Задание на курсовой проект
по дисциплине
«Объектно-ориентированное программирование»

Студенту группы М412 — В. Д. Панкову.

Тема проекта: «Разработка настольного приложения для организации взаимодействия объектов библиотечной информационной системы».

Исходные данные: предметная область «Библиотека», иерархия сущностей, графическое Desktop-приложение, локальное хранение в SQLite через Room 3 и применение шаблонов проектирования.

Проект должен содержать:
1. анализ предметной области и формулировку функционала
2. разработку иерархии классов сущностей, управляющих и интерфейсных классов
3. обоснование шаблонов проектирования и построение диаграмм
4. разработку Desktop-приложения и тестовых сценариев
5. тестирование с фиксацией фактических результатов
6. оформление пояснительной записки и презентации

Срок сдачи законченного проекта: не указан в доступных материалах проекта.
Руководитель проекта: Е. О. Шумова.
Дата выдачи задания: не указана в доступных материалах проекта.
Формальное согласование темы: в доступных материалах проекта не зафиксировано.

Содержание

1. Постановка задачи

. 5

1.1. Анализ предметной области 5

1.2. Формулировка функционала 5

2. Разработка классов

. 7

2.1. Классы сущностей 7

2.2. Управляющие классы 9

2.3. Интерфейсные классы 10

2.4. Используемые паттерны проектирования 10

2.4.1. Repository 10

2.4.2. MVVM 11

2.4.3. Strategy 12

2.4.4. Factory 12

2.4.5. Observer 13

2.4.6. State 14

2.4.7. Facade 14

2.5. Общая архитектура и схема хранения 15

3. Разработка приложения

. 17

3.1. Разработка интерфейса приложения 17

3.2. Реализация методов классов 18

3.3. Контрольный сценарий приложения 19

4. Тестирование

. 20

core/src/commonMain/kotlin/domain/LibraryEntity.kt 24

core/src/commonMain/kotlin/domain/DomainModels.kt 25
core/src/commonMain/kotlin/domain/LibraryEntityFactory.kt 27
core/src/commonMain/kotlin/domain/LibraryService.kt 28
core/src/commonMain/kotlin/domain/LoanPolicy.kt 28
core/src/commonMain/kotlin/application/RoomLibraryService.kt 29
core/src/commonMain/kotlin/persistence/Database.kt 30
core/src/commonMain/kotlin/persistence/LibraryDao.kt 30
core/src/commonMain/kotlin/persistence/LibraryDatabase.kt 34
core/src/commonMain/kotlin/persistence/Records.kt 35
core/src/commonMain/kotlin/persistence/Repositories.kt 36
core/src/jvmMain/kotlin/persistence/DatabaseJvm.kt 40
app/src/commonMain/kotlin/App.kt 40
app/src/commonMain/kotlin/di/ApplicationContainer.kt 41
app/src/commonMain/kotlin/ui/LibraryViewModels.kt 41
app/src/commonMain/kotlin/ui/LibraryScreen.kt 46
app/src/jvmMain/kotlin/main.kt 51

ВВЕДЕНИЕ

Объектно-ориентированное программирование позволяет представить предметную область как систему взаимодействующих объектов с ясными ответственностями. В библиотечной информационной системе такими объектами являются читатели, сотрудники, издания, филиалы и факты выдачи. Для завершённой реализации важно не только описать классы, но и связать их с пользовательскими сценариями, хранением данных и проверяемыми инвариантами.

Цель курсового проекта — разработать и протестировать настольное графическое приложение, демонстрирующее взаимодействие иерархии классов библиотечной предметной области с применением принципов ООП и шаблонов проектирования. Финальная реализация представляет собой приложение Desktop JVM на Kotlin и Compose Desktop. Сохранение выполняется в локальной SQLite-базе с использованием Room 3 и BundledSQLiteDriver.

Для достижения цели выполнены следующие задачи:

- проанализирована предметная область и выделены сущности;
- построены иерархии Person — Reader и Librarian, Publication — Book и Periodical;
- реализована многозначная связь Reader и Publication через ассоциативную сущность Loan;
- разделены сущностные, управляющие и интерфейсные классы;
- применены семь паттернов: Repository, MVVM, Strategy, Factory, Observer, State и Facade;
- обеспечены непрозрачные UUID, постоянное хранение, транзакционные выдача и возврат, а также идемпотентная загрузка демонстрационных данных;
- выполнены JVM unit- и integration-тесты, результат сборки сохранён в артефактах.

1. Постановка задачи

1.1. Анализ предметной области

Проект моделирует обслуживание читателей в библиотеке. Основные понятия предметной области и их роли следующие:

- читатель — зарегистрированный пользователь, которому могут выдаваться издания;
- библиотекарь — сотрудник, оформляющий выдачу и связанный с филиалом;
- издание — единица фонда; в проекте представлены книга и периодическое издание;
- филиал — место хранения фонда;
- выдача — зафиксированный факт передачи издания читателю с датами и состоянием.

Существительные предметной области преобразуются в сущностные классы, а глаголы — в операции приложения: зарегистрировать читателя, добавить издание, сохранить, выдать, вернуть, просмотреть данные и загрузить демонстрационный набор. Идентификаторы сущностей представлены строками, но их значения являются непрозрачными: UuidIdGenerator создаёт UUID внутри приложения, а пользователь выбирает читателя и издание по предметным признакам.

Многозначная зависимость является центральной связью модели. Один Reader может иметь много Loan, а одно Publication может фигурировать во многих Loan. Каждая выдача ссылается на одного читателя, одно издание и одного библиотекаря. В базе эта связь выражена внешними ключами readerId, publicationId и librarianId таблицы loans.

Приложение ограничено настольным JVM-сценарием. В core и app объявлена цель jvm(), точка входа создаёт окно Compose Desktop, а зависимости распределены между доменным ядром и графическим модулем.

1.2. Формулировка функционала

Функциональность финальной версии определяется исходным кодом и тестами.

Группа	Реализованные возможности
Читатели	создание, поиск по имени, отображение и удаление читателей; валидация полей; сохранение через репозиторий

Группа	Реализованные возможности
Издания	создание книг и периодических изданий, поиск по названию или автору, отображение остатка и удаление; выбор филиала
Выдачи	выбор читателя и издания; проверка библиотекаря, политики и доступности; атомарное оформление; отображение выдач и возврат
Целостность	непрозрачные UUID; внешние ключи; запрет удаления объектов с историей выдач; возврат только один раз
Администрирование	транзакционная загрузка связанного демонстрационного набора; повторный вызов без дубликатов; безопасная очистка всех таблиц
Хранение	запись в SQLite, закрытие и повторное открытие Room-базы с восстановлением доменных объектов

Интерфейс предоставляет четыре вкладки: Читатели, Издания, Выдачи и Архитектура. На первых двух доступны создание, поиск, просмотр и удаление; на третьей выбираются читатель и доступное издание, после чего выполняются операции Выдать и Вернуть. Четвёртая вкладка показывает ООП-иерархии, связь многие-ко-многим, слои и семь паттернов, а также содержит явные команды добавления демонстрационных данных и очистки. Технические идентификаторы в поля формы не выводятся.

2. Разработка классов

2.1. Классы сущностей

Корневой контракт LibraryEntity содержит только непрозрачный id. Через него образованы две иерархии: Person с реализациями Reader и Librarian, а также Publication с реализациями Book и Periodical. LibraryBranch и Loan также реализуют LibraryEntity, но не являются подтипами Person или Publication.

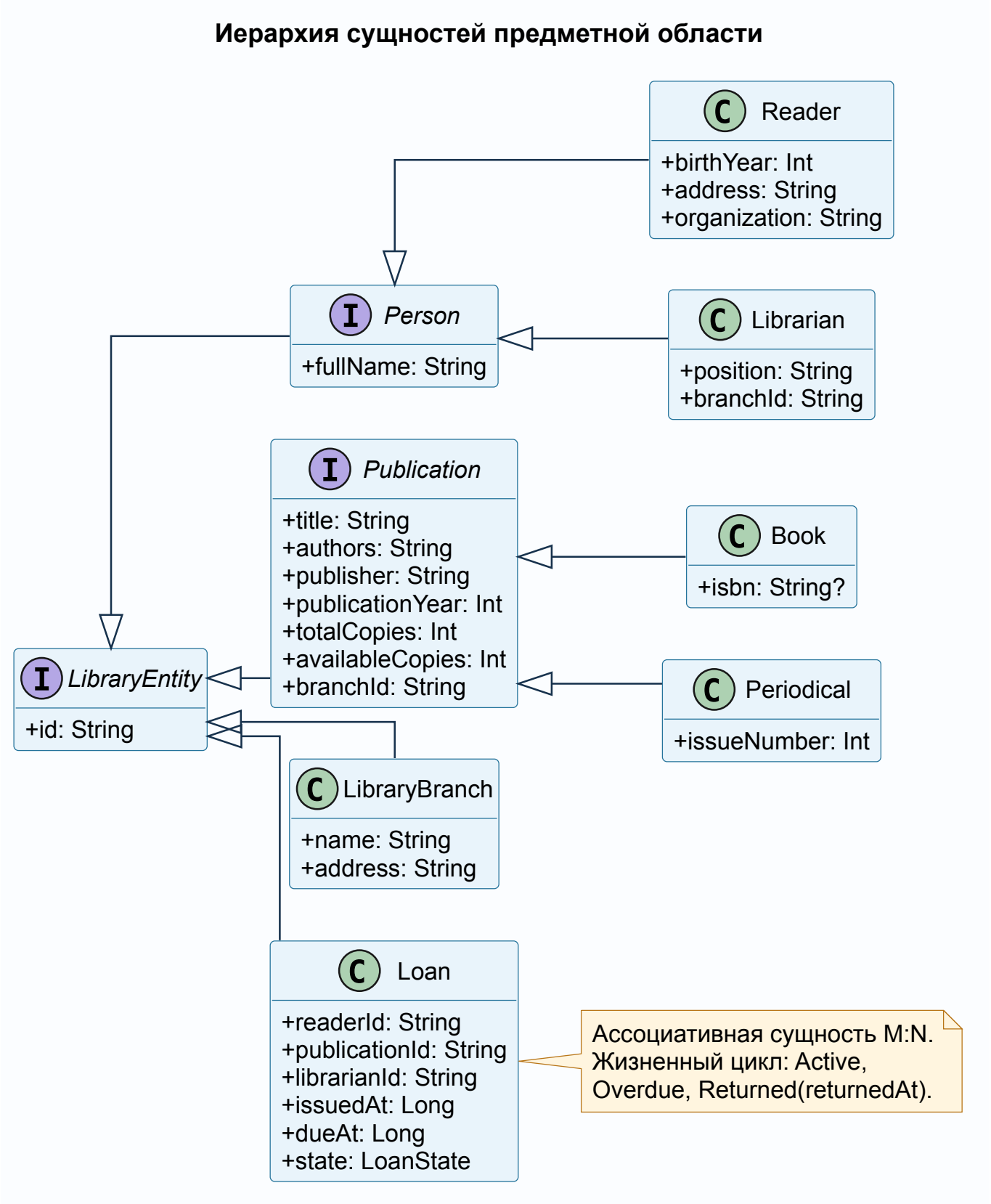


Рисунок 1. Иерархия классов сущностей

Класс	Категория	Основные поля	Ответственность
LibraryEntity	интерфейс	id	единый контракт идентификации

Класс	Категория	Основные поля	Ответственность
Person	интерфейс	fullName	общий контракт человека
Reader	сущность	id, fullName, birthYear, address, organization	зарегистрированный читатель и проверка данных
Librarian	сущность	id, fullName, position, branchId	сотрудник филиала, оформляющий выдачу
Publication	интерфейс	title, authors, publisher, publicationYear, totalCopies, availableCopies, branchId	общий контракт фонда
Book	сущность	общие поля Publication и isbn	книга с инвариантами фонда
Periodical	сущность	общие поля Publication и issueNumber	выпуск периодического издания
LibraryBranch	сущность	id, name, address	место хранения фонда
Loan	сущность	readerId, publicationId, librarianId, issuedAt, dueAt, state	ассоциативная связь и жизненный цикл выдачи

В LoanState определены состояния Active, Overdue и Returned с датой возврата. Переход returnAt переводит активную или просроченную выдачу в Returned; повторный возврат завершается доменной ошибкой. LoanPolicy задаёт проверку допустимости выдачи и срок возврата: в DefaultLoanPolicy срок зависит от типа публикации, а для читателя действует предел активных выдач.

2.2. Управляющие классы

Управляющий слой координирует сущности, хранилище и представление.

- LibraryService — доменный интерфейс фасада с операциями выдачи, возврата, защищённого удаления, повторно безопасной загрузки и транзакционной очистки. RoomLibraryService зависит от доменного LibraryRepositoryBundle, проверяет связанные объекты, вызывает LoanPolicy и передаёт атомарные операции адаптерам.
- RoomRepositoryBundle реализует доменный LibraryRepositoryBundle и объединяет адаптеры читателей, сотрудников, филиалов, изданий и выдач. Интерфейсы возвращают только доменные объекты и Flow; типы Room Record не выходят из persistence-слоя.

- LibraryEntityFactory создаёт валидные Reader, Librarian, LibraryBranch, Book, Periodical и Loan, скрывая генерацию UUID и проверку конструкторов.
- ReadersViewModel, PublicationsViewModel, LoansViewModel и AdministrationViewModel принимают события интерфейса, формируют UiState, выполняют поиск и валидацию и запускают корутины для операций. LoansViewModel раз в минуту обновляет просроченные выдачи, поэтому открытый экран не сохраняет устаревшее состояние Active после наступления dueAt.
- ApplicationContainer является явной точкой сборки зависимостей: создаёт базу, набор репозиториев и RoomLibraryService, а при закрытии освобождает базу.

2.3. Интерфейсные классы

Интерфейсный слой реализован composable-компонентами модуля app. main.kt создаёт ApplicationContainer, открывает Window и передаёт контейнер в App; загрузка демонстрационных данных выполняется только явной кнопкой. Компонент App создаёт ViewModel и передаёт их в LibraryScreen. Внутри экрана находятся панели ReadersPane, PublicationsPane, LoansPane и ArchitecturePane.

Компоненты отвечают за компоновку элементов и пользовательские события, но не выполняют SQL и не меняют инвентарь напрямую. Состояния загрузки, пустого списка и ошибки отображаются через CollectionUiState. Список выдач получает имена читателей и названия публикаций, сопоставляя их с идентификаторами, которые остаются внутренними для приложения.

2.4. Используемые паттерны проектирования

Ниже каждый из семи паттернов рассмотрен отдельно. Диаграммы подключены из соответствующих финальных SVG-файлов каталога docs/images.

2.4.1. Repository

Repository отделяет доменную часть от конкретного хранилища. Интерфейсы ReaderRepository, PublicationRepository и LoanRepository описывают операции над доменными объектами, а классы с префиксом Room отображают записи Room и делегируют запросы LibraryDao. Благодаря этому ViewModel и сервис не знают SQL-представление таблиц.

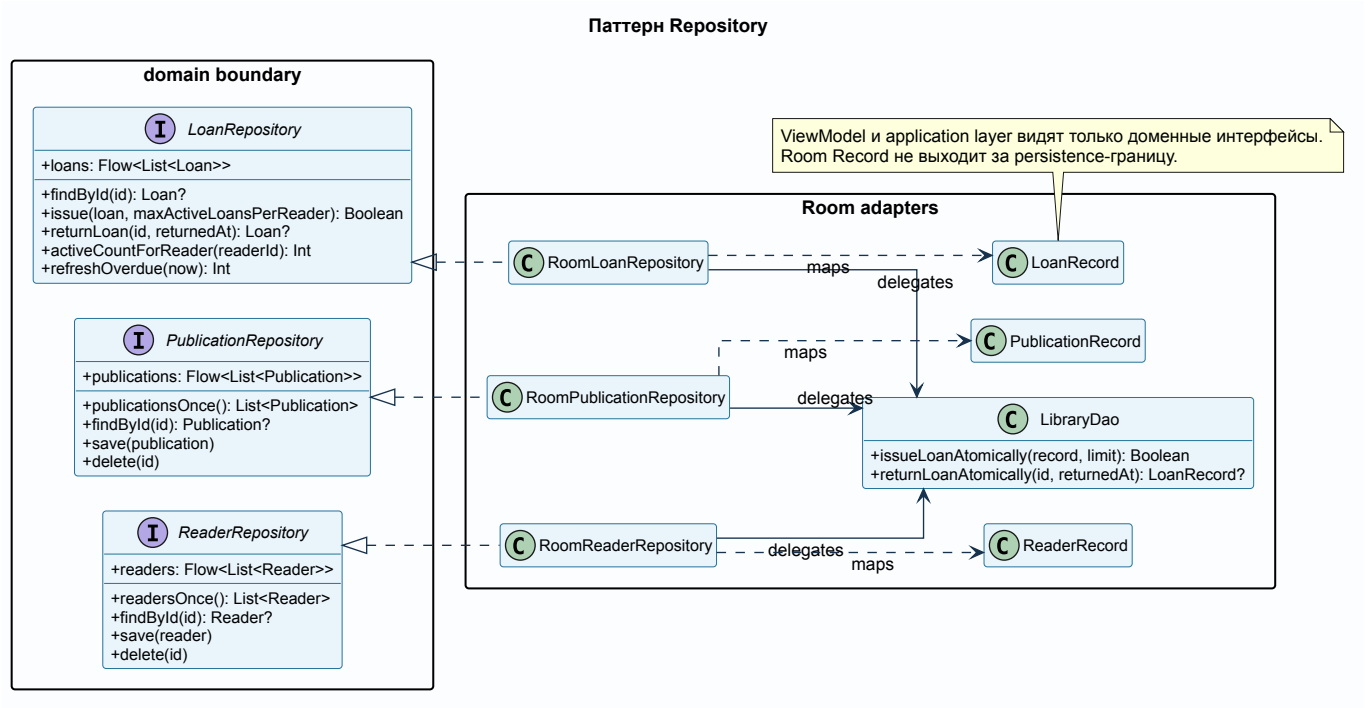


Рисунок 2. Паттерн Repository: доменные интерфейсы и Room-адаптеры

2.4.2. MVVM

MVVM разделяет Compose-представление, состояние и сценарии. LibraryScreen и панели отображают CollectionUiState, ViewModel принимает действия пользователя, а RoomLibraryService и репозитории выполняют операции. ViewModel можно проверять отдельно от окна, что подтверждено тестами валидации и операций.

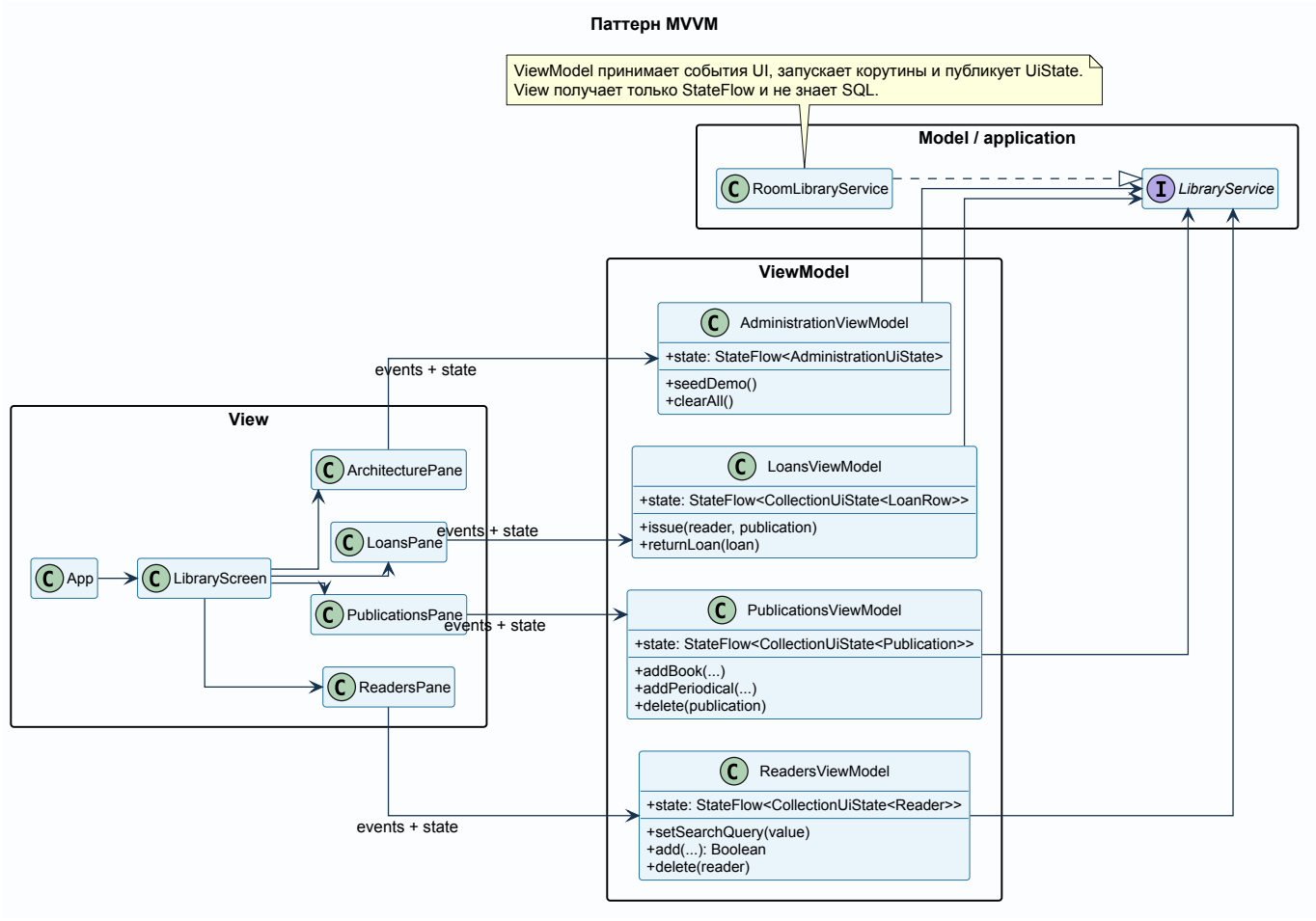


Рисунок 3. Паттерн MVVM: Compose, ViewModel и application layer

2.4.3. Strategy

Strategy представлен интерфейсом LoanPolicy. RoomLibraryService получает политику через конструктор и использует её для решения canIssue и вычисления dueAt. DefaultLoanPolicy применяет разные сроки для Book и Periodical, поэтому правило выдачи можно заменить без изменения фасада и хранилища.

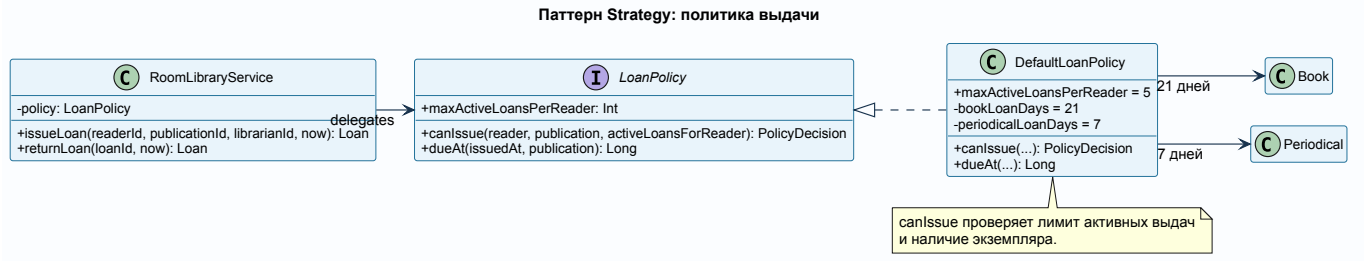


Рисунок 4. Паттерн Strategy: LoanPolicy и правила выдачи

2.4.4. Factory

LibraryEntityFactory инкапсулирует создание валидных сущностей и получение UUID через IdGenerator. В seedDemo фабрика используется для построения филиала, биб-

лиотекаря, двух читателей, книги, периодического издания и выдачи. Затем записи передаются в одну транзакцию Room. Клиенту не требуется знать порядок конструирования объектов.

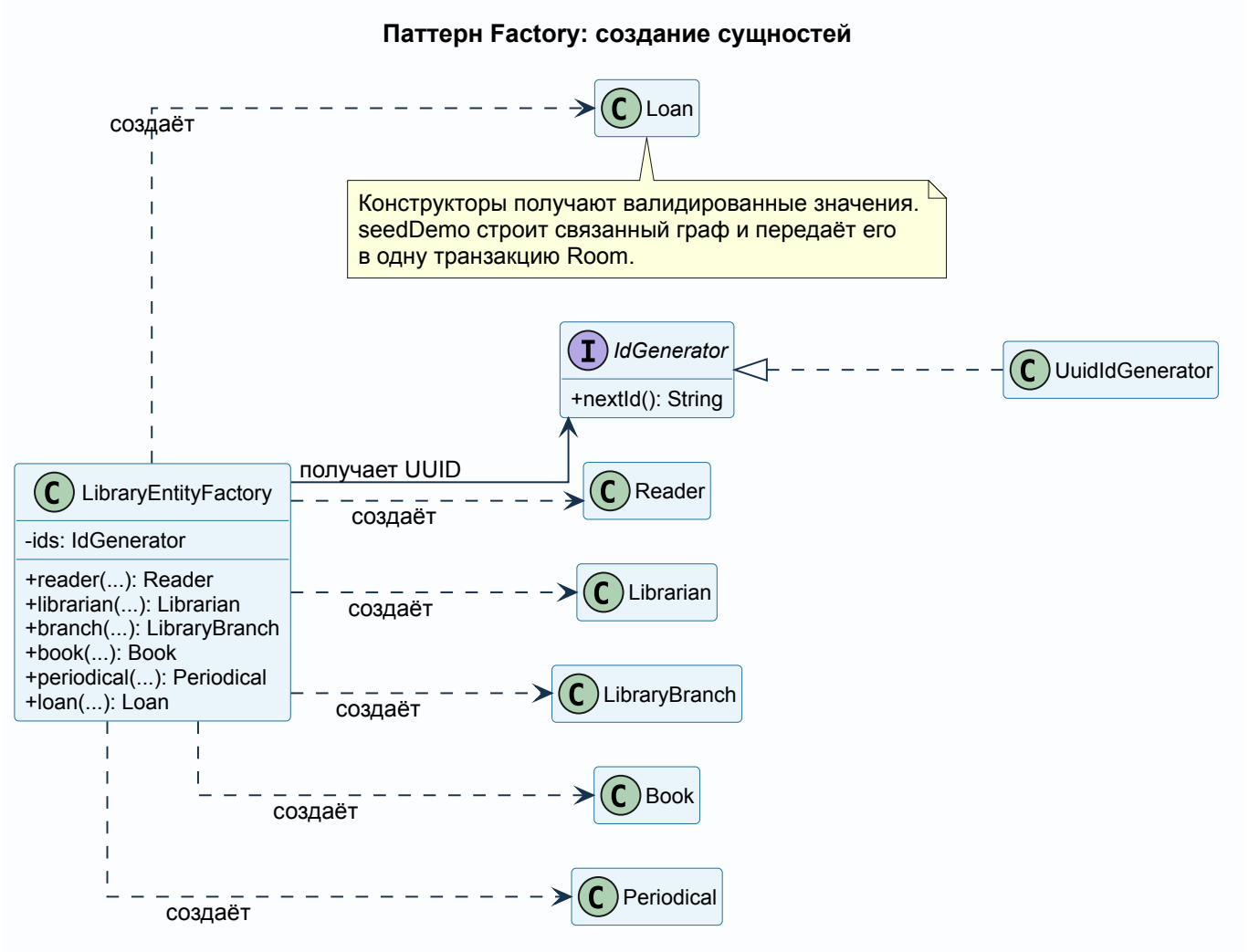


Рисунок 5. Паттерн Factory: создание сущностей и демонстрационного графа

2.4.5. Observer

В роли источника изменений выступают потоки списков, которые возвращает DAO. Репозитории преобразуют записи в доменные объекты, а ViewModel переводит поток в состояние коллекции через stateIn. Compose получает состояние через collectAsState. Изменение записи автоматически распространяется на наблюдающий экран; просроченные активные записи обновляются при подписке и периодически, пока экран выдач открыт.

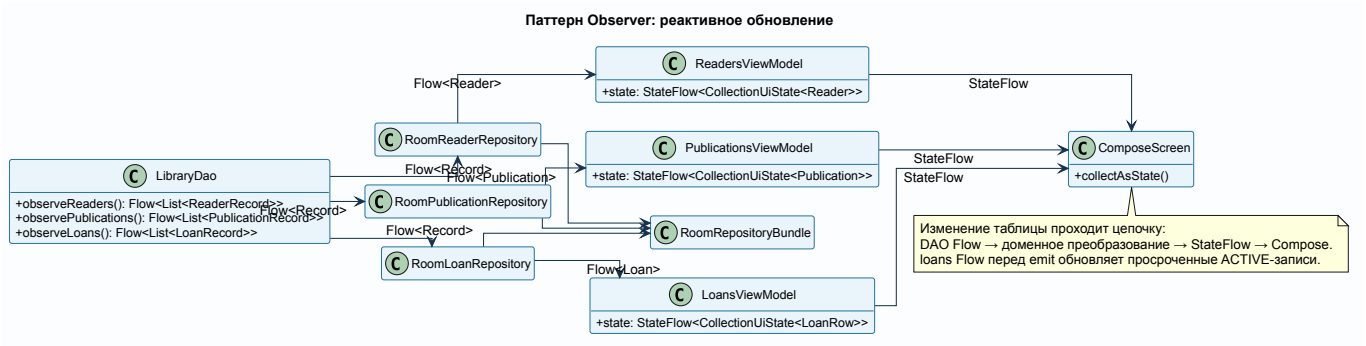


Рисунок 6. Паттерн Observer: Room Flow — StateFlow — Compose

2.4.6. State

Loan хранит объект состояния LoanState. Состояния Active, Overdue и Returned выражают жизненный цикл выдачи, а функция returnAt разрешает переход к возврату только из первых двух состояний. При повторном вызове возврата DAO также возвращает уже завершённую запись, не увеличивая инвентарь второй раз.

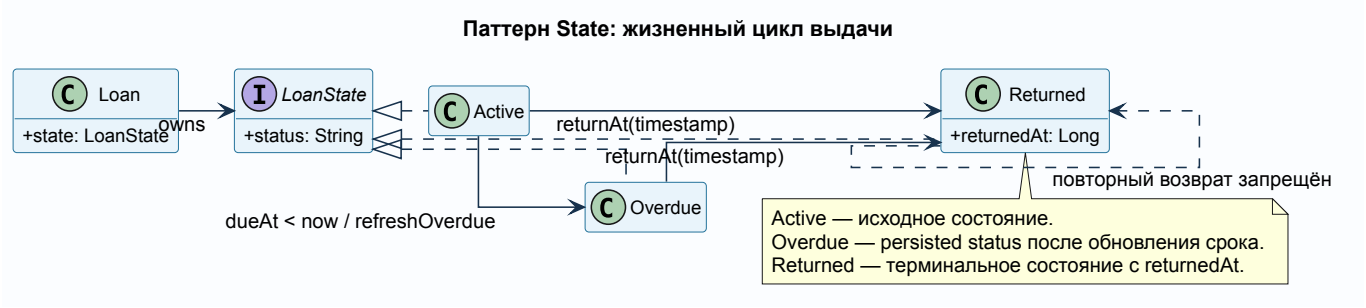


Рисунок 7. Паттерн State: состояния и переходы Loan

2.4.7. Facade

LibraryService предоставляет клиенту короткие сценарии выдачи и возврата. RoomLibraryService скрывает поиск читателя, публикации и библиотекаря, применение LoanPolicy, генерацию идентификатора и вызов транзакционного репозитория. Тем самым ViewModel не координирует несколько DAO самостоятельно.

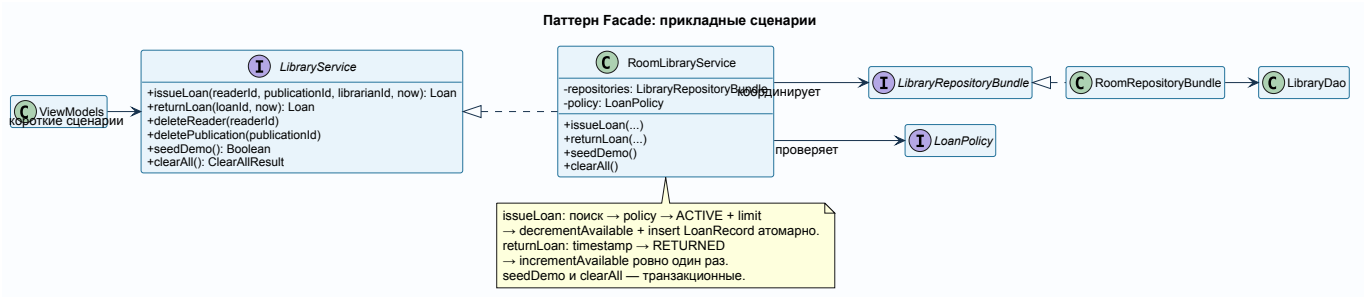


Рисунок 8. Паттерн Facade: LibraryService и координация сценария

2.5. Общая архитектура и схема хранения

Архитектура разделена на пять логических уровней. Доменный слой содержит сущности, состояния, политики и фабрику. Persistence-слой содержит записи Room, DAO, базу и преобразования. Репозитории образуют границу между этими слоями. Application-слой реализует фасад и сценарии. UI-слой содержит Compose Desktop, ViewModel и контейнер зависимостей.

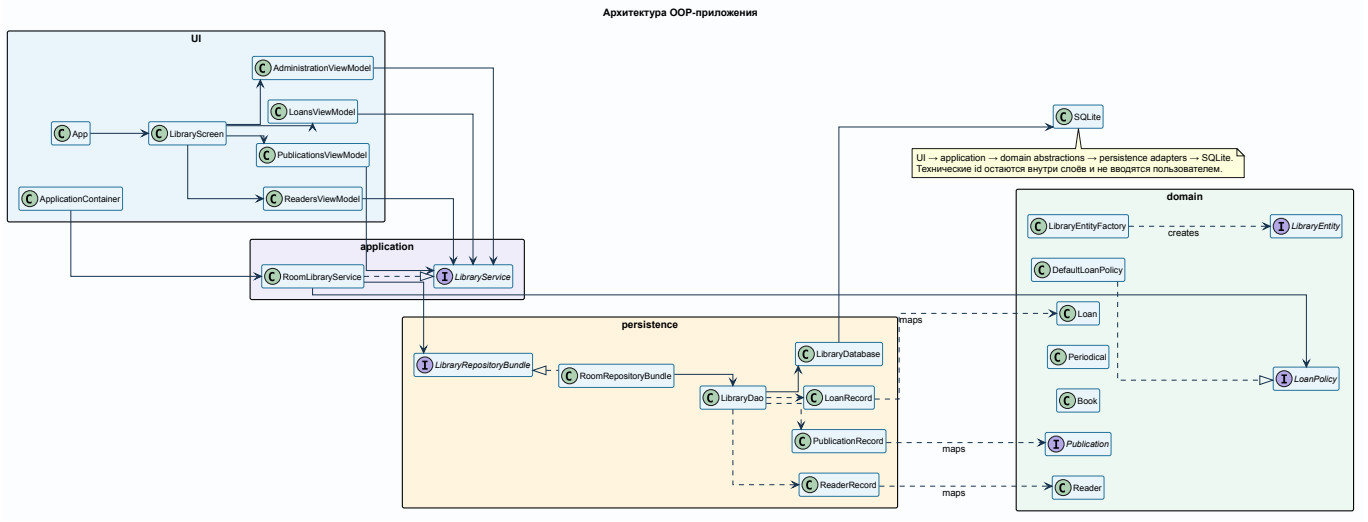


Рисунок 9. Общая архитектурная диаграмма финального приложения

LibraryDatabase объявляет пять таблиц: readers, librarians, branches, publications и loans. Таблица publications хранит дискриминатор kind, библиографические поля и количества экземпляров. Для loans определены внешние ключи на читателя, публикацию и библиотекаря, а также индексы по этим полям.

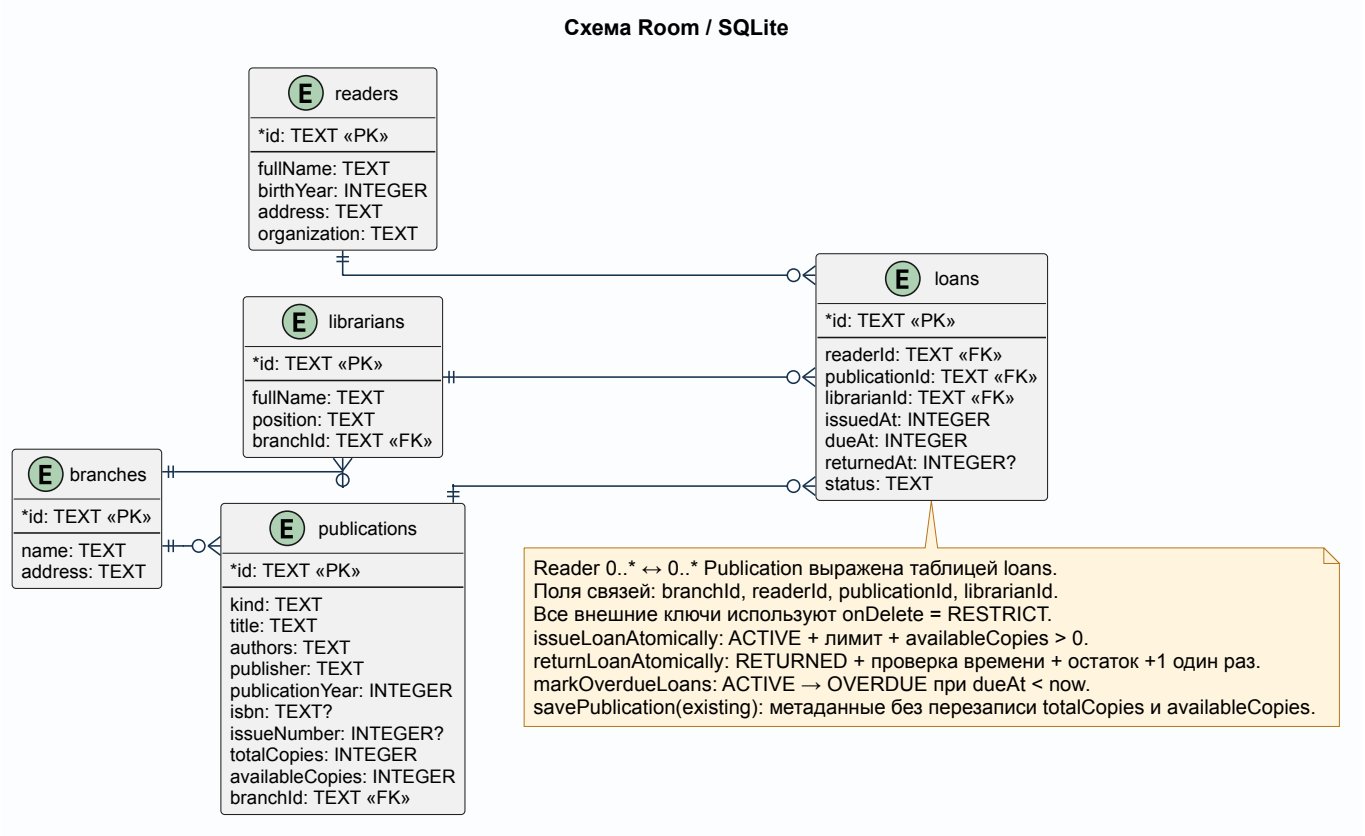


Рисунок 10. Схема Room/SQLite и внешние ключи

Выдача выполняется внутри транзакционного метода: сначала проверяются статус ACTIVE и лимит активных выдач читателя, затем условный SQL-запрос уменьшает availableCopies и вставляется LoanRecord; при отказе изменённые данные откатываются. Возврат проверяет timestamp, помечает запись как RETURNED и увеличивает доступное количество ровно один раз. Наблюдение выдач передаёт сохранённое обновление просрочки, а удаление читателя и издания защищено от любой истории выдач. Загрузка демонстрационных данных транзакционна: если выдача невозможна, записи подготовленного набора откатываются. Проверка непустой таблицы филиалов делает повторный вызов seedDemo идемпотентным.

3. Разработка приложения

3.1. Разработка интерфейса приложения

Структура меню финального приложения соответствует четырём вкладкам главного экрана.

Вкладка	Компонент	Назначение
Читатели	ReadersPane	поиск по имени, ввод полей, добавление, список, удаление и состояния ошибки или загрузки
Издания	PublicationsPane	поиск по названию или автору, выбор филиала, добавление Book или Periodical, доступность и удаление
Выдачи	LoansPane	выбор читателя, выбор публикации с доступным экземпляром, выдача, список выдач и возврат
Архитектура	ArchitecturePane	иерархии, связь многие-ко-многим, конвейер, семь паттернов, добавление демонстрационных данных и очистка

Пользователь не вводит id, readerId, publicationId или librarianId. В панели выдач выбираются объекты по имени и названию, после чего ViewModel передаёт внутренние UUID сервису. Для публикаций доступны поля ISBN книги и номера выпуска периодического издания; состав объекта определяется нажатием соответствующей кнопки.

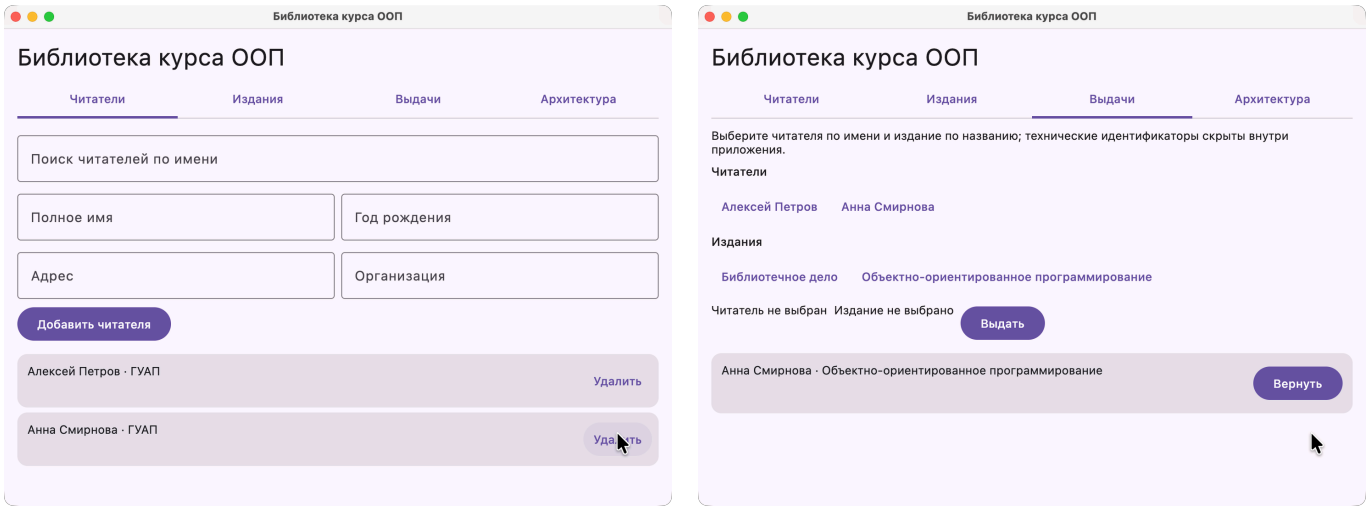


Рисунок 11. Финальный интерфейс: управление читателями и сценарий выдачи

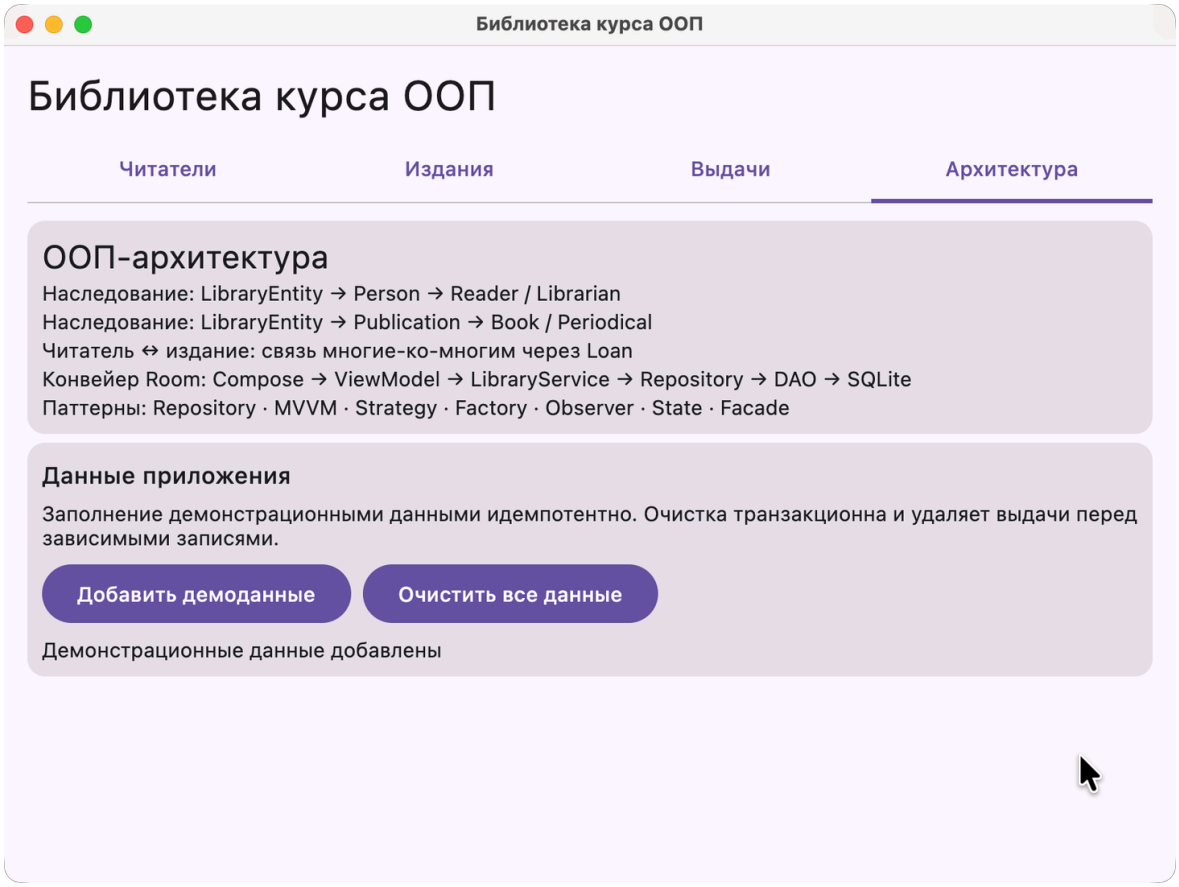


Рисунок 12. Вкладка «Архитектура» и явные операции демонстрационных данных

3.2. Реализация методов классов

Методы распределены по ответственностям. Конструкторы доменных классов проверяют непустые строки, допустимые годы, положительное общее количество и диапазон availableCopies. ViewModel проверяет преобразование текстовых полей до запуска фоновой операции. Репозитории выполняют отображение Record и доменного объекта.

Ключевой метод issueLoan в RoomLibraryService работает так:

1. получает читателя, публикацию и библиотекаря по внутренним идентификаторам;
2. передаёт читателя, публикацию и число активных выдач в LoanPolicy;
3. создаёт Loan с UUID, временем выдачи и рассчитанным сроком;
4. вызывает атомарную выдачу, которая уменьшает остаток и вставляет запись;
5. сообщает об ошибке, если подходящей копии нет.

Метод returnLoan делегирует returnLoanAtomically. DAO получает текущую запись, не меняет уже возвращённую выдачу, а для активной записи обновляет статус и увеличивает остаток. Поэтому повторный вызов идемпотентен.

3.3. Контрольный сценарий приложения

При запуске main создаёт ApplicationContainer и открывает окно «Библиотека курса ООП»; скрытая автоматическая загрузка данных не выполняется. На вкладке «Архитектура» пользователь нажимает «Добавить демоданные»: в базу транзакционно попадают филиал, библиотекарь, два читателя, книга, периодическое издание и одна активная выдача. Повторное нажатие не создаёт дубликаты. Команда «Очистить все данные» в одной транзакции удаляет сначала выдачи, затем зависимые публикации, читателей, сотрудников и филиалы.

Пользовательский сценарий после открытия окна: загрузить демонстрационные данные; найти читателя по имени и публикацию по названию или автору; добавить новые объекты; в разделе «Выдачи» выбрать читателя и доступное издание; выполнить «Выдать»; убедиться, что доступное количество уменьшилось; выполнить «Вернуть» и убедиться, что количество восстановилось. Попытка удалить объект со связанной выдачей показывает понятную ошибку; ошибочный ввод отклоняется до записи.

4. Тестирование

Тестирование выполнено на JVM и включает доменные проверки, интеграцию с реальной Room/SQLite-базой, транзакционные инварианты, связь многие-ко-многим, поиск, удаление, очистку, контрольный сценарий и ViewModel. Фактический итог зафиксирован в artifacts/test-results.txt: 35 тестов, из них 17 в core и 18 в app; ошибки, ошибки выполнения и пропуски отсутствуют. Полная сборка зафиксирована в artifacts/build-results.txt.

Тестовый класс	**Тестов**	**Проверяемое поведение**
DomainRewriteTest	4	UUID-идентификаторы, валидация домена, переход LoanState и различие правил LoanPolicy
RoomPersistenceTest	2	запись, закрытие, повторное открытие и восстановление; атомарная выдача, возврат и повторный возврат
RoomInvariantTest	2	запрет удаления связанных объектов; откат полностью неуспешной транзакции загрузки
ReviewRegressionTest	5	лимит активных выдач, сохранение общего и доступного количества при обновлении метаданных, строгий ACTIVE-инвариант, обновление просрочки и проверка даты возврата
ManyToManyTest	1	две выдачи одного издания разным читателям и корректное уменьшение инвентаря
ControlScenarioTest	1	успешная загрузка, идемпотентный повтор и ожидаемый граф данных
RoomAdministrationTest	2	защищённое удаление, доменная граница загрузки и транзакционная очистка с подсчётом удалённых записей
ViewModelValidationTest	1	отклонение некорректной формы без записи в репозиторий
ViewModelOperationsTest	4	поиск читателей и изданий, удаление и отображение ошибок сервисных операций
ViewModelLifecycleAndErrorTest	13	отмена области ViewModel, распространение CancellationException, независимые operation/refresh errors, восстановление busy/error-state

Основной сохранённый командный прогон:

./gradlew clean test build –no-daemon

Результат: BUILD SUCCESSFUL. 35 тестов, 0 ошибок, 0 ошибок выполнения и 0 пропусков.

Сфокусированные прогоны охватывают persistence, application и ViewModel. RoomPersistenceTest проверяет повторное открытие SQLite; ReviewRegressionTest — лимит, инвентарь, состояния и время; RoomAdministrationTest — защищённое удаление и очистку; ViewModelOperationsTest — фильтрацию и ошибки; ViewModelLifecycleAndErrorTest — жизненный цикл, отмену, busy-state, просрочку и независимые operation/refresh errors. ManyToManyTest подтверждает две выдачи одного издания разным читателям.

В логе сборки отмечено предупреждение текущего JDK о native access для Bundled SQLite. Оно не является ошибкой и не изменило результат BUILD SUCCESSFUL. Финальное окно было запущено через :app:run и визуально проверено; снимки интерфейса приведены в разделе 3.1.

Контрольный сценарий покрывает повторную безопасную загрузку демонстрационных данных и ожидаемый граф объектов. Административные операции выполняются транзакционно: сначала удаляются выдачи, затем зависимые сущности, а результат очистки возвращается в состояние интерфейса. Проверки ViewModel дополнительно подтверждают отказ некорректной формы до обращения к репозиторию, передачу ошибок и отмену области после уничтожения экрана.

Таким образом, тестовый комплект проверяет не только отдельные функции, но и согласованность доменного слоя, Room persistence, application service и Compose UI. Фактические результаты воспроизводятся командами из README и приложенных summary-файлов; все 35 тестов завершились успешно.

ЗАКЛЮЧЕНИЕ

В курсовом проекте разработано настольное JVM-приложение для учёта читателей, публикаций, филиалов и выдач. Цель достигнута: предметная область представлена иерархией LibraryEntity, Person — Reader и Librarian, Publication — Book и Periodical; Loan и LibraryBranch дополняют модель.

Связь Reader и Publication реализована через Loan, что позволяет хранить историю операций и поддерживать множество выдач с каждой стороны. Технические идентификаторы генерируются как UUID и не являются частью пользовательского ввода. Room 3 и SQLite обеспечивают восстановление данных, а внешние ключи и транзакции защищают целостность. Выдача, возврат и демонстрационная загрузка согласованы с инвентарём, причём повторный возврат и повторная загрузка данных идемпотентны.

Семь паттернов имеют конкретные точки применения: Repository скрывает Room, MVVM разделяет экран и состояние, Strategy инкапсулирует LoanPolicy, Factory создаёт валидные объекты, Observer передаёт изменения от Flow к StateFlow, State описывает жизненный цикл Loan, а Facade предоставляет сценарии LibraryService.

Фактический прогон ./gradlew clean test build –no-daemon завершился успешно: 35 тестов, 17 в core и 18 в app, 0 ошибок, 0 ошибок выполнения и 0 пропусков. Финальный интерфейс поддерживает человекочитаемый поиск, создание, обновление метаданных с сохранением общего количества и доступного остатка, защищённое удаление, выдачу с лимитом активных займов, возврат с проверкой времени, сохранённое и периодическое обновление просрочки, независимое восстановление operation/refresh errors, корректное распространение отмены корутин без зависшего busy-state, явную повторно безопасную загрузку и транзакционную очистку. Дальнейшее развитие может включать отдельный редактор количества экземпляров с явными инвентарными операциями и расширенную отчётность по истории выдач.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Шумова Е. О. Объектно-ориентированное программирование: методические указания к выполнению курсового проекта. Санкт-Петербург: СПбГУАП, 2026.
2. README.md проекта: описание финальной архитектуры, Room 3, модулей и команд проверки.
3. context/TASK.md, context/context.yaml, context/requirements-checklist.md, context/TARGET_ARCHITECTURE.md — исходные требования и целевая архитектура проекта.
4. Kotlin documentation [Электронный ресурс]. URL: https://kotlinlang.org/docs/home.html (дата обращения: 02.09.2026).
5. Compose Multiplatform documentation [Электронный ресурс]. URL: https://www.jetbrains.com/compose-multiplatform/ (дата обращения: 02.09.2026).
6. AndroidX Room documentation [Электронный ресурс]. URL: https://developer.android.com/training/data-storage/room (дата обращения: 02.09.2026).
7. Gamma E., Helm R., Johnson R., Vlissides J. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994.
8. Исходный код и тесты текущего проекта: каталоги core/src, app/src, core/schemas и docs/images.
9. Сохранённые результаты: artifacts/test-results.txt, artifacts/build-results.txt, artifacts/tdd-room-rewrite.md и artifacts/code-results.md.

ПРИЛОЖЕНИЕ А. ПОЛНЫЕ ЛИСТИНГИ PRODUCTION-ФАЙЛОВ

Ниже приведены полные текущие тексты всех production-файлов, относящихся к домену, application, persistence, DI, UI и точкам входа. Каждый листинг читается Typst непосредственно из файла проекта. Тестовые, сгенерированные Room/KSP и вспомогательные копии не включаются.

core/src/commonMain/kotlin/domain/LibraryEntity.kt

```
@file:OptIn(kotlin.uuid.ExperimentalUuidApi::class)

package domain

import kotlin.uuid.Uuid

/** Root contract for all domain entities. IDs are generated internally and opaque to users. */
interface LibraryEntity {
    val id: String
}

interface IdGenerator {
    fun nextId(): String
}

class UuidIdGenerator : IdGenerator {
    override fun nextId(): String = Uuid.random().toString()
}

interface Person : LibraryEntity {
    val fullName: String
}

interface Publication : LibraryEntity {
    val title: String
    val authors: String
    val publisher: String
    val publicationYear: Int
    val totalCopies: Int
    val availableCopies: Int
    val branchId: String
}

fun validateText(value: String, field: String) {
    val label = mapOf(
        "fullName" to "Имя",
        "address" to "Адрес",
        "organization" to "Организация",
        "position" to "Должность",
        "branchId" to "Филиал",
        "name" to "Название",
        "title" to "Название",
        "authors" to "Авторы",
        "publisher" to "Издатель",
        "readerId" to "Читатель",
        "publicationId" to "Издание",
        "librarianId" to "Библиотекарь",
    )[field] ?: field
```

```
    require(value.isNotBlank()) { "$label не должно быть пустым" }
}

fun validatePublicationCopies(totalCopies: Int, availableCopies: Int) {
    require(totalCopies > 0) { "Общее число экземпляров должно быть положительным" }
    require(availableCopies in 0..totalCopies) { "Доступное число экземпляров должно быть от 0 до общего числа" }
}
```

core/src/commonMain/kotlin/domain/DomainModels.kt

```
package domain

/** A person registered as a library reader. */
data class Reader(
    override val id: String,
    override val fullName: String,
    val birthYear: Int,
    val address: String,
    val organization: String,
) : Person {
    init {
        validateText(fullName, "fullName")
        require(birthYear in 1800..2100) { "Год рождения вне поддерживаемого диапазона" }
        validateText(address, "address")
        validateText(organization, "organization")
    }
}

/** A library employee who can issue publications. */
data class Librarian(
    override val id: String,
    override val fullName: String,
    val position: String,
    val branchId: String,
) : Person {
    init {
        validateText(fullName, "fullName")
        validateText(position, "position")
        validateText(branchId, "branchId")
    }
}

/** A physical or logical library location. */
data class LibraryBranch(
    override val id: String,
    val name: String,
    val address: String,
) : LibraryEntity {
    init {
        validateText(name, "name")
        validateText(address, "address")
    }
}

/** A book in the collection. */
data class Book(
    override val id: String,
    override val title: String,
    override val authors: String,
    override val publisher: String,
    override val publicationYear: Int,
    override val totalCopies: Int,
```

```
    override val availableCopies: Int,
    override val branchId: String,
 val isbn: String?,
 ) : Publication {
    init {
        validateText(title, "title")
        validateText(authors, "authors")
        validateText(publisher, "publisher")
        validateText(branchId, "branchId")
        require(publicationYear in 1450..2100) { "Год издания вне поддерживаемого диапазона" }
        validatePublicationCopies(totalCopies, availableCopies)
    }
}

/** A periodical issue in the collection. */
data class Periodical(
    override val id: String,
    override val title: String,
    override val authors: String,
    override val publisher: String,
    override val publicationYear: Int,
    override val totalCopies: Int,
    override val availableCopies: Int,
    override val branchId: String,
    val issueNumber: Int,
) : Publication {
    init {
        validateText(title, "title")
        validateText(authors, "authors")
        validateText(publisher, "publisher")
        validateText(branchId, "branchId")
        require(publicationYear in 1450..2100) { "Год издания вне поддерживаемого диапазона" }
        require(issueNumber > 0) { "Номер выпуска должен быть положительным" }
        validatePublicationCopies(totalCopies, availableCopies)
    }
}

/** State pattern for a loan lifecycle. */
sealed interface LoanState {
    val status: String

    data object Active : LoanState {
        override val status: String = "ACTIVE"
        override fun toString(): String = status
    }

    data object Overdue : LoanState {
        override val status: String = "OVERDUE"
        override fun toString(): String = status
    }

    data class Returned(val returnedAt: Long) : LoanState {
        override val status: String = "RETURNED"
    }
}

fun LoanState.returnAt(timestamp: Long): LoanState = when (this) {
    LoanState.Active, LoanState.Overdue -> LoanState.Returned(timestamp)
    is LoanState.Returned -> error("Выдача уже возвращена")
}
```

```
data class Loan(
    override val id: String,
    val readerId: String,
    val publicationId: String,
    val librarianId: String,
    val issuedAt: Long,
    val dueAt: Long,
    val state: LoanState = LoanState.Active,
) : LibraryEntity {
    init {
        validateText(readerId, "readerId")
        validateText(publicationId, "publicationId")
        validateText(librarianId, "librarianId")
        require(dueAt ⩾ issuedAt) { "Дата возврата не может быть раньше даты выдачи" }
    }
}
```

core/src/commonMain/kotlin/domain/LibraryEntityFactory.kt

```
package domain

class LibraryEntityFactory(private val ids: IdGenerator = UuidIdGenerator()) {
    fun reader(fullName: String, birthYear: Int, address: String, organization: String): Reader =
        Reader(ids.nextId(), fullName, birthYear, address, organization)

    fun librarian(fullName: String, position: String, branchId: String): Librarian =
        Librarian(ids.nextId(), fullName, position, branchId)

    fun branch(name: String, address: String): LibraryBranch =
        LibraryBranch(ids.nextId(), name, address)

    fun book(
        title: String,
        authors: String,
        publisher: String,
        publicationYear: Int,
        totalCopies: Int,
        branchId: String,
        isbn: String? = null,
    ): Book = Book(
        ids.nextId(), title, authors, publisher, publicationYear,
        totalCopies, totalCopies, branchId, isbn,
    )

    fun periodical(
        title: String,
        authors: String,
        publisher: String,
        publicationYear: Int,
        issueNumber: Int,
        totalCopies: Int,
        branchId: String,
    ): Periodical = Periodical(
        ids.nextId(), title, authors, publisher, publicationYear,
        totalCopies, totalCopies, branchId, issueNumber,
    )

    fun loan(
        readerId: String,
        publicationId: String,
        librarianId: String,
        issuedAt: Long,
```

```
        dueAt: Long,
    ): Loan = Loan(ids.nextId(), readerId, publicationId, librarianId, issuedAt, dueAt)
}
```

core/src/commonMain/kotlin/domain/LibraryService.kt

```
package domain

/** Domain-facing facade; the persistence-backed implementation lives in the application layer. */
interface LibraryService {
    suspend fun issueLoan(readerId: String, publicationId: String, librarianId: String, now: Long): Loan
    suspend fun returnLoan(loanId: String, now: Long): Loan
    suspend fun deleteReader(readerId: String)
    suspend fun deletePublication(publicationId: String)
    suspend fun seedDemo(): Boolean
    suspend fun clearAll(): ClearAllResult
}

data class DemoData(
    val branch: LibraryBranch,
    val librarian: Librarian,
    val readers: List<Reader>,
    val publications: List<Publication>,
    val loan: Loan,
)

data class ClearAllResult(
    val loans: Int,
    val publications: Int,
    val readers: Int,
    val librarians: Int,
    val branches: Int,
) {
    val totalDeleted: Int
        get() = loans + publications + readers + librarians + branches
}
```

core/src/commonMain/kotlin/domain/LoanPolicy.kt

```
package domain

interface LoanPolicy {
    val maxActiveLoansPerReader: Int
        get() = Int.MAX_VALUE

    fun canIssue(reader: Reader, publication: Publication, activeLoansForReader: Int): PolicyDecision
    fun dueAt(issuedAt: Long, publication: Publication): Long
}

data class PolicyDecision(val isAllowed: Boolean, val reason: String? = null)

class DefaultLoanPolicy(
    override val maxActiveLoansPerReader: Int = 5,
    private val bookLoanDays: Long = 21,
    private val periodicalLoanDays: Long = 7,
) : LoanPolicy {
    init {
        require(maxActiveLoansPerReader > 0)
        require(bookLoanDays > 0)
        require(periodicalLoanDays > 0)
    }
```

```
    override fun canIssue(
        reader: Reader,
        publication: Publication,
        activeLoansForReader: Int,
    ): PolicyDecision = when {
        activeLoansForReader >= maxActiveLoansPerReader ->
            PolicyDecision(false, "Читатель достиг лимита активных выдач")
        publication.availableCopies <= 0 ->
            PolicyDecision(false, "У издания нет доступных экземпляров")
        else -> PolicyDecision(true)
    }

    override fun dueAt(issuedAt: Long, publication: Publication): Long {
        val days = if (publication is Periodical) periodicalLoanDays else bookLoanDays
        return issuedAt + days * MILLIS_PER_DAY
    }

    private companion object {
        const val MILLIS_PER_DAY = 86_400_000L
    }
}
```

core/src/commonMain/kotlin/application/RoomLibraryService.kt

```
package application

import domain.DefaultLoanPolicy
import domain.DemoData
import domain.IdGenerator
import domain.LibraryEntityFactory
import domain.LibraryService
import domain.Loan
import domain.LoanPolicy
import domain.LoanState
import domain.UuidIdGenerator
import persistence.LibraryRepositoryBundle

class RoomLibraryService(
    private val repositories: LibraryRepositoryBundle,
    private val policy: LoanPolicy = DefaultLoanPolicy(),
    private val ids: IdGenerator = UuidIdGenerator(),
) : LibraryService {
    override suspend fun issueLoan(readerId: String, publicationId: String, librarianId: String, now: Long): Loan {
        val reader = repositories.readers.findById(readerId) ?: error("Читатель не найден")
        val publication = repositories.publications.findById(publicationId) ?: error("Издание не найдено")
        repositories.librarians.findById(librarianId) ?: error("Библиотекарь не найден")
        val decision = policy.canIssue(reader, publication, repositories.loans.activeCountForReader(readerId))
        if (!decision.isAllowed) error(decision.reason ?: "Выдача запрещена")
        val loan = Loan(
            id = ids.nextId(),
            readerId = readerId,
            publicationId = publicationId,
            librarianId = librarianId,
            issuedAt = now,
            dueAt = policy.dueAt(now, publication),
            state = LoanState.Active,
        )
        if (!repositories.loans.issue(loan, policy.maxActiveLoansPerReader)) {
            error("Выдача запрещена или у издания нет доступных экземпляров")
        }
        return loan
    }
```

```
    override suspend fun returnLoan(loanId: String, now: Long): Loan =
        repositories.loans.returnLoan(loanId, now) ?: error("Выдача не найдена")

    override suspend fun deleteReader(readerId: String) {
        repositories.readers.delete(readerId)
    }

    override suspend fun deletePublication(publicationId: String) {
        repositories.publications.delete(publicationId)
    }

    override suspend fun seedDemo(): Boolean {
        val factory = LibraryEntityFactory(ids)
        val branch = factory.branch("Центральная библиотека", "Невский проспект, 28")
        val librarian = factory.librarian("Елена Волкова", "Старший библиотекарь", branch.id)
        val reader = factory.reader("Анна Смирнова", 1995, "Санкт-Петербург", "ГУАП")
        val secondReader = factory.reader("Алексей Петров", 1994, "Санкт-Петербург", "ГУАП")
        val book = factory.book("Объектно-ориентированное программирование", "Е. О. Шумова", "ГУАП", 2026, 2, branch.id,
"978-5-00000-001-0")
        val periodical = factory.periodical("Библиотечное дело", "Редакционная коллегия", "Профессия", 2025, 1, 2, branch.id)
        val loan = factory.loan(reader.id, book.id, librarian.id, 1_700_000_000_000L, 1_701_814_400_000L)
        return repositories.insertDemo(
            DemoData(
                branch = branch,
                librarian = librarian,
                readers = listOf(reader, secondReader),
                publications = listOf(book, periodical),
                loan = loan,
            ),
        )
    }

    override suspend fun clearAll() = repositories.clearAll()
}
```

core/src/commonMain/kotlin/persistence/Database.kt

```
package persistence

import androidx.room3.RoomDatabase
import androidx.sqlite.driver.bundled.BundledSQLiteDriver
import kotlinx.coroutines.Dispatchers

fun buildDatabase(builder: RoomDatabase.Builder<LibraryDatabase>): LibraryDatabase =
    builder
        .setDriver(BundledSQLiteDriver())
        .setQueryCoroutineContext(Dispatchers.IO)
        .build()
```

core/src/commonMain/kotlin/persistence/LibraryDao.kt

```
package persistence

import androidx.room3.Dao
import androidx.room3.Insert
import androidx.room3.OnConflictStrategy
import androidx.room3.Query
import androidx.room3.Transaction
import androidx.room3.Upsert
import kotlinx.coroutines.flow.Flow
```

```
@Dao
interface LibraryDao {
    @Query("SELECT * FROM readers ORDER BY fullName")
    fun observeReaders(): Flow<List<ReaderRecord>>

    @Query("SELECT * FROM readers ORDER BY fullName")
    suspend fun getReaders(): List<ReaderRecord>

    @Query("SELECT * FROM readers WHERE id = :id")
    suspend fun getReader(id: String): ReaderRecord?

    @Upsert
    suspend fun saveReader(record: ReaderRecord)

    @Query("DELETE FROM readers WHERE id = :id")
    suspend fun deleteReader(id: String): Int

    @Query("SELECT COUNT(*) FROM loans WHERE readerId = :id AND returnedAt IS NULL")
    suspend fun countActiveLoansForReader(id: String): Int

    @Query("SELECT COUNT(*) FROM loans WHERE readerId = :id")
    suspend fun countLoansForReader(id: String): Int

    @Transaction
    suspend fun deleteReaderIfUnused(id: String): Int {
        if (getReader(id) == null) return -1
        if (countLoansForReader(id) > 0) return 0
        return deleteReader(id)
    }

    @Query("SELECT * FROM librarians ORDER BY fullName")
    fun observeLibrarians(): Flow<List<LibrarianRecord>>

    @Query("SELECT * FROM librarians ORDER BY fullName")
    suspend fun getLibrarians(): List<LibrarianRecord>

    @Query("SELECT * FROM librarians WHERE id = :id")
    suspend fun getLibrarian(id: String): LibrarianRecord?

    @Upsert
    suspend fun saveLibrarian(record: LibrarianRecord)

    @Query("SELECT * FROM branches ORDER BY name")
    fun observeBranches(): Flow<List<BranchRecord>>

    @Query("SELECT * FROM branches ORDER BY name")
    suspend fun getBranches(): List<BranchRecord>

    @Query("SELECT COUNT(*) FROM branches")
    suspend fun countBranches(): Int

    @Upsert
    suspend fun saveBranch(record: BranchRecord)

    @Query("SELECT * FROM publications ORDER BY title")
    fun observePublications(): Flow<List<PublicationRecord>>

    @Query("SELECT * FROM publications ORDER BY title")
    suspend fun getPublications(): List<PublicationRecord>

    @Query("SELECT * FROM publications WHERE id = :id")
```

```
suspend fun getPublication(id: String): PublicationRecord?

@Insert(onConflict = OnConflictStrategy.IGNORE)
suspend fun insertPublicationIfAbsent(record: PublicationRecord): Long

@Query("""
    UPDATE publications SET
        kind = :kind,
        title = :title,
        authors = :authors,
        publisher = :publisher,
        publicationYear = :publicationYear,
        isbn = :isbn,
        issueNumber = :issueNumber,
        branchId = :branchId
    WHERE id = :id
""")
suspend fun updatePublicationMetadata(
    id: String,
    kind: String,
    title: String,
    authors: String,
    publisher: String,
    publicationYear: Int,
    isbn: String?,
    issueNumber: Int?,
    branchId: String,
): Int

@Transaction
suspend fun savePublication(record: PublicationRecord) {
    if (insertPublicationIfAbsent(record) == -1L) {
        updatePublicationMetadata(
            record.id,
            record.kind,
            record.title,
            record.authors,
            record.publisher,
            record.publicationYear,
            record.isbn,
            record.issueNumber,
            record.branchId,
        )
    }
}

@Query("UPDATE publications SET availableCopies = availableCopies - 1 WHERE id = :publicationId AND availableCopies > 0")
suspend fun decrementAvailable(publicationId: String): Int

@Query("UPDATE publications SET availableCopies = availableCopies + 1 WHERE id = :publicationId AND availableCopies < totalCopies")
suspend fun incrementAvailable(publicationId: String): Int

@Query("DELETE FROM publications WHERE id = :id")
suspend fun deletePublication(id: String): Int

@Query("SELECT COUNT(*) FROM loans WHERE publicationId = :id AND returnedAt IS NULL")
suspend fun countActiveLoansForPublication(id: String): Int

@Query("SELECT COUNT(*) FROM loans WHERE publicationId = :id")
suspend fun countLoansForPublication(id: String): Int
```

```
@Transaction
suspend fun deletePublicationIfUnused(id: String): Int {
    if (getPublication(id) == null) return -1
    if (countLoansForPublication(id) > 0) return 0
    return deletePublication(id)
}

@Query("SELECT * FROM loans ORDER BY issuedAt DESC")
fun observeLoans(): Flow<List<LoanRecord>>

@Query("SELECT * FROM loans ORDER BY issuedAt DESC")
suspend fun getLoans(): List<LoanRecord>

@Query("SELECT * FROM loans WHERE id = :id")
suspend fun getLoan(id: String): LoanRecord?

@Query("SELECT COUNT(*) FROM loans WHERE readerId = :readerId AND returnedAt IS NULL")
suspend fun countActiveLoans(readerId: String): Int

@Query("UPDATE loans SET status = 'OVERDUE' WHERE status = 'ACTIVE' AND returnedAt IS NULL AND dueAt < :now")
suspend fun markOverdueLoans(now: Long): Int

@Insert
suspend fun insertLoan(record: LoanRecord)

@Query("UPDATE loans SET returnedAt = :returnedAt, status = 'RETURNED' WHERE id = :loanId AND returnedAt IS NULL")
suspend fun markReturned(loanId: String, returnedAt: Long): Int

@Transaction
suspend fun issueLoanAtomically(record: LoanRecord, maxActiveLoansPerReader: Int): Boolean {
    if (record.status != "ACTIVE" || record.returnedAt != null) return false
    if (countActiveLoans(record.readerId) >= maxActiveLoansPerReader) return false
    if (decrementAvailable(record.publicationId) != 1) return false
    insertLoan(record)
    return true
}

@Transaction
suspend fun issueLoanAtomically(record: LoanRecord): Boolean =
    issueLoanAtomically(record, Int.MAX_VALUE)

@Transaction
suspend fun returnLoanAtomically(loanId: String, returnedAt: Long): LoanRecord? {
    val current = getLoan(loanId) ?: return null
    require(returnedAt >= current.issuedAt) { "Дата возврата не может быть раньше даты выдачи" }
    if (current.returnedAt != null) return current
    if (markReturned(loanId, returnedAt) != 1) return getLoan(loanId)
    if (incrementAvailable(current.publicationId) != 1) {
        error("Нарушен инвариант складского остатка издания")
    }
    return getLoan(loanId)
}

@Query("SELECT COUNT(*) FROM loans")
suspend fun countLoans(): Int

@Query("SELECT COUNT(*) FROM publications")
suspend fun countPublications(): Int

@Query("SELECT COUNT(*) FROM readers")
suspend fun countReaders(): Int
```

```
    @Query("SELECT COUNT(*) FROM librarians")
    suspend fun countLibrarians(): Int

    @Query("DELETE FROM loans")
    suspend fun deleteAllLoans(): Int

    @Query("DELETE FROM publications")
    suspend fun deleteAllPublications(): Int

    @Query("DELETE FROM readers")
    suspend fun deleteAllReaders(): Int

    @Query("DELETE FROM librarians")
    suspend fun deleteAllLibrarians(): Int

    @Query("DELETE FROM branches")
    suspend fun deleteAllBranches(): Int

    @Transaction
    suspend fun clearAllData(): ClearAllCounts {
        val counts = ClearAllCounts(countLoans(), countPublications(), countReaders(), countLibrarians(), countBranches())
        // Loans must be removed first because all three entity relationships are RESTRICT.
        deleteAllLoans()
        deleteAllPublications()
        deleteAllReaders()
        deleteAllLibrarians()
        deleteAllBranches()
        return counts
    }

    @Transaction
    suspend fun insertDemoData(
        branch: BranchRecord,
        librarian: LibrarianRecord,
        readers: List<ReaderRecord>,
        publications: List<PublicationRecord>,
        loan: LoanRecord,
    ): Boolean {
        if (countBranches() > 0) return false
        saveBranch(branch)
        saveLibrarian(librarian)
        readers.forEach { saveReader(it) }
        publications.forEach { savePublication(it) }
        if (!issueLoanAtomically(loan)) error("Не удалось оформить демонстрационную выдачу; транзакция отменена")
        return true
    }
}
```

core/src/commonMain/kotlin/persistence/LibraryDatabase.kt

```
package persistence

import androidx.room3.ConstructedBy
import androidx.room3.Database
import androidx.room3.RoomDatabase
import androidx.room3.RoomDatabaseConstructor

@Database(
    entities = [ReaderRecord::class, LibrarianRecord::class, BranchRecord::class, PublicationRecord::class, LoanRecord::class],
    version = 1,
    exportSchema = true,
```

```
)
@ConstructedBy(LibraryDatabaseConstructor::class)
abstract class LibraryDatabase : RoomDatabase() {
    abstract fun libraryDao(): LibraryDao
}

@Suppress("KotlinNoActualForExpect")
expect object LibraryDatabaseConstructor : RoomDatabaseConstructor<LibraryDatabase> {
    override fun initialize(): LibraryDatabase
}
```

core/src/commonMain/kotlin/persistence/Records.kt

```
package persistence

import androidx.room3.Entity
import androidx.room3.ForeignKey
import androidx.room3.Index
import androidx.room3.PrimaryKey

@Entity(tableName = "readers")
data class ReaderRecord(
    @PrimaryKey val id: String,
    val fullName: String,
    val birthYear: Int,
    val address: String,
    val organization: String,
)

@Entity(
    tableName = "librarians",
    foreignKeys = [ForeignKey(
        entity = BranchRecord::class,
        parentColumns = ["id"],
        childColumns = ["branchId"],
        onDelete = ForeignKey.RESTRICT,
    )],
    indices = [Index("branchId")],
)
data class LibrarianRecord(
    @PrimaryKey val id: String,
    val fullName: String,
    val position: String,
    val branchId: String,
)

@Entity(tableName = "branches")
data class BranchRecord(
    @PrimaryKey val id: String,
    val name: String,
    val address: String,
)

@Entity(
    tableName = "publications",
    foreignKeys = [ForeignKey(
        entity = BranchRecord::class,
        parentColumns = ["id"],
        childColumns = ["branchId"],
        onDelete = ForeignKey.RESTRICT,
    )],
    indices = [Index("branchId")],
```

```
)
data class PublicationRecord(
    @PrimaryKey val id: String,
    val kind: String,
    val title: String,
    val authors: String,
    val publisher: String,
    val publicationYear: Int,
    val isbn: String?,
    val issueNumber: Int?,
    val totalCopies: Int,
    val availableCopies: Int,
    val branchId: String,
)

@Entity(
    tableName = "loans",
    foreignKeys = [
        ForeignKey(
            entity = ReaderRecord::class,
            parentColumns = ["id"],
            childColumns = ["readerId"],
            onDelete = ForeignKey.RESTRICT,
        ),
        ForeignKey(
            entity = PublicationRecord::class,
            parentColumns = ["id"],
            childColumns = ["publicationId"],
            onDelete = ForeignKey.RESTRICT,
        ),
        ForeignKey(
            entity = LibrarianRecord::class,
            parentColumns = ["id"],
            childColumns = ["librarianId"],
            onDelete = ForeignKey.RESTRICT,
        ),
    ],
    indices = [Index("readerId"), Index("publicationId"), Index("librarianId")],
)
data class LoanRecord(
    @PrimaryKey val id: String,
    val readerId: String,
    val publicationId: String,
    val librarianId: String,
    val issuedAt: Long,
    val dueAt: Long,
    val returnedAt: Long?,
    val status: String,
)

data class ClearAllCounts(
    val loans: Int,
    val publications: Int,
    val readers: Int,
    val librarians: Int,
    val branches: Int,
)
```

core/src/commonMain/kotlin/persistence/Repositories.kt

```
package persistence
```

```
import domain.Book
import domain.ClearAllResult
import domain.DemoData
import domain.LibraryBranch
import domain.Librarian
import domain.Loan
import domain.LoanState
import domain.Periodical
import domain.Publication
import domain.Reader
import androidx.sqlite.SQLiteException
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.emitAll
import kotlinx.coroutines.flow.map

interface ReaderRepository {
    val readers: Flow<List<Reader>>
    suspend fun readersOnce(): List<Reader>
    suspend fun findById(id: String): Reader?
    suspend fun save(reader: Reader)
    suspend fun delete(id: String)
}

interface LibrarianRepository {
    val librarians: Flow<List<Librarian>>
    suspend fun findById(id: String): Librarian?
    suspend fun save(librarian: Librarian)
}

interface BranchRepository {
    val branches: Flow<List<LibraryBranch>>
    suspend fun findById(id: String): LibraryBranch?
    suspend fun save(branch: LibraryBranch)
}

interface PublicationRepository {
    val publications: Flow<List<Publication>>
    suspend fun publicationsOnce(): List<Publication>
    suspend fun findById(id: String): Publication?
    suspend fun save(publication: Publication)
    suspend fun delete(id: String)
}

interface LoanRepository {
    val loans: Flow<List<Loan>>
    suspend fun findById(id: String): Loan?
    suspend fun issue(loan: Loan): Boolean
    suspend fun issue(loan: Loan, maxActiveLoansPerReader: Int): Boolean = issue(loan)
    suspend fun returnLoan(id: String, returnedAt: Long): Loan?
    suspend fun activeCountForReader(readerId: String): Int
    suspend fun refreshOverdue(now: Long): Int = 0
}

interface LibraryAdministrationRepository {
    suspend fun insertDemo(data: DemoData): Boolean
    suspend fun clearAll(): ClearAllResult
}

interface LibraryRepositoryBundle : LibraryAdministrationRepository {
    val readers: ReaderRepository
    val librarians: LibrarianRepository
```

```
    val branches: BranchRepository
    val publications: PublicationRepository
    val loans: LoanRepository
}

class ActiveLoanDeletionException(message: String, cause: Throwable? = null) : IllegalStateException(message, cause)
class InventoryException(message: String) : IllegalStateException(message)

class RoomReaderRepository(private val dao: LibraryDao) : ReaderRepository {
    override val readers: Flow<List<Reader>> = dao.observeReaders().map { records -> records.map { readerRecordToDomain(it) } }
    override suspend fun readersOnce(): List<Reader> = dao.getReaders().map { readerRecordToDomain(it) }
    override suspend fun findById(id: String): Reader? = dao.getReader(id)?.let { readerRecordToDomain(it) }
    override suspend fun save(reader: Reader) = dao.saveReader(reader.toRecord())
    override suspend fun delete(id: String) {
        try {
            when (dao.deleteReaderIfUnused(id)) {
                -1 -> Unit
                0 -> throw ActiveLoanDeletionException("Нельзя удалить читателя: есть история или активные выдачи")
            }
        } catch (error: ActiveLoanDeletionException) {
            throw error
        } catch (error: SQLiteException) {
            throw ActiveLoanDeletionException("Нельзя удалить читателя: связи с выдачами изменились", error)
        }
    }
}

class RoomLibrarianRepository(private val dao: LibraryDao) : LibrarianRepository {
    override val librarians: Flow<List<Librarian>> = dao.observeLibrarians().map { records -> records.map
{ librarianRecordToDomain(it) } }
    override suspend fun findById(id: String): Librarian? = dao.getLibrarian(id)?.let { librarianRecordToDomain(it) }
    override suspend fun save(librarian: Librarian) = dao.saveLibrarian(librarian.toRecord())
}

class RoomBranchRepository(private val dao: LibraryDao) : BranchRepository {
    override val branches: Flow<List<LibraryBranch>> = dao.observeBranches().map { records -> records.map { branchRecordToDomain(it) } }
    override suspend fun findById(id: String): LibraryBranch? = dao.getBranches().firstOrNull { it.id == id }?.let
{ branchRecordToDomain(it) }
    override suspend fun save(branch: LibraryBranch) = dao.saveBranch(branch.toRecord())
}

class RoomPublicationRepository(private val dao: LibraryDao) : PublicationRepository {
    override val publications: Flow<List<Publication>> = dao.observePublications().map { records -> records.map
{ publicationRecordToDomain(it) } }
    override suspend fun publicationsOnce(): List<Publication> = dao.getPublications().map { publicationRecordToDomain(it) }
    override suspend fun findById(id: String): Publication? = dao.getPublication(id)?.let { publicationRecordToDomain(it) }
    override suspend fun save(publication: Publication) = dao.savePublication(publication.toRecord())
    override suspend fun delete(id: String) {
        try {
            when (dao.deletePublicationIfUnused(id)) {
                -1 -> Unit
                0 -> throw ActiveLoanDeletionException("Нельзя удалить издание: есть история или активные выдачи")
            }
        } catch (error: ActiveLoanDeletionException) {
            throw error
        } catch (error: SQLiteException) {
            throw ActiveLoanDeletionException("Нельзя удалить издание: связи с выдачами изменились", error)
        }
    }
}
```

```
class RoomLoanRepository(private val dao: LibraryDao) : LoanRepository {
    @OptIn(kotlin.time.ExperimentalTime::class)
    override val loans: Flow<List<Loan>> = kotlinx.coroutines.flow.flow {
        dao.markOverdueLoans(kotlin.time.Clock.System.now().toEpochMilliseconds())
        emitAll(dao.observeLoans())
    }.map { records -> records.map { loanRecordToDomain(it) } }

    @OptIn(kotlin.time.ExperimentalTime::class)
    override suspend fun findById(id: String): Loan? {
        dao.markOverdueLoans(kotlin.time.Clock.System.now().toEpochMilliseconds())
        return dao.getLoan(id)?.let { loanRecordToDomain(it) }
    }

    override suspend fun issue(loan: Loan): Boolean = issue(loan, Int.MAX_VALUE)
    override suspend fun issue(loan: Loan, maxActiveLoansPerReader: Int): Boolean {
        if (loan.state != LoanState.Active) return false
        require(maxActiveLoansPerReader > 0)
        return dao.issueLoanAtomically(loan.toRecord(), maxActiveLoansPerReader)
    }

    override suspend fun returnLoan(id: String, returnedAt: Long): Loan? =
        dao.returnLoanAtomically(id, returnedAt)?.let { loanRecordToDomain(it) }

    override suspend fun activeCountForReader(readerId: String): Int = dao.countActiveLoans(readerId)
    override suspend fun refreshOverdue(now: Long): Int = dao.markOverdueLoans(now)
}

class RoomRepositoryBundle(val database: LibraryDatabase) : LibraryRepositoryBundle {
    private val dao = database.libraryDao()
    override val readers = RoomReaderRepository(dao)
    override val librarians = RoomLibrarianRepository(dao)
    override val branches = RoomBranchRepository(dao)
    override val publications = RoomPublicationRepository(dao)
    override val loans = RoomLoanRepository(dao)

    override suspend fun insertDemo(data: DemoData): Boolean = dao.insertDemoData(
        data.branch.toRecord(),
        data.librarian.toRecord(),
        data.readers.map { it.toRecord() },
        data.publications.map { it.toRecord() },
        data.loan.toRecord(),
    )

    override suspend fun clearAll(): ClearAllResult = dao.clearAllData().let {
        ClearAllResult(it.loans, it.publications, it.readers, it.librarians, it.branches)
    }

    /** Compatibility entry point for persistence-focused rollback tests. */
    suspend fun insertDemoData(
        branch: BranchRecord,
        librarian: LibrarianRecord,
        readers: List<ReaderRecord>,
        publications: List<PublicationRecord>,
        loan: LoanRecord,
    ): Boolean = dao.insertDemoData(branch, librarian, readers, publications, loan)
}

private fun Reader.toRecord() = ReaderRecord(id, fullName, birthYear, address, organization)
private fun readerRecordToDomain(record: Reader) = record
private fun readerRecordToDomain(record: ReaderRecord) = Reader(record.id, record.fullName, record.birthYear, record.address,
record.organization)
```

```
private fun Librarian.toRecord() = LibrarianRecord(id, fullName, position, branchId)
private fun librarianRecordToDomain(record: LibrarianRecord) = Librarian(record.id, record.fullName, record.position, record.branchId)
private fun LibraryBranch.toRecord() = BranchRecord(id, name, address)
private fun branchRecordToDomain(record: BranchRecord) = LibraryBranch(record.id, record.name, record.address)
private fun Publication.toRecord() = when (this) {
    is Book → PublicationRecord(id, "BOOK", title, authors, publisher, publicationYear, isbn, null, totalCopies, availableCopies,
branchId)
    is Periodical → PublicationRecord(id, "PERIODICAL", title, authors, publisher, publicationYear, null, issueNumber, totalCopies,
availableCopies, branchId)
    else → error("Неподдерживаемый тип издания")
}
private fun publicationRecordToDomain(record: PublicationRecord): Publication = when (record.kind) {
    "BOOK" → Book(record.id, record.title, record.authors, record.publisher, record.publicationYear, record.totalCopies,
record.availableCopies, record.branchId, record.isbn)
    "PERIODICAL" → Periodical(record.id, record.title, record.authors, record.publisher, record.publicationYear, record.totalCopies,
record.availableCopies, record.branchId, record.issueNumber ?: 1)
    else → error("Неизвестный тип издания: ${record.kind}")
}
private fun Loan.toRecord() = LoanRecord(id, readerId, publicationId, librarianId, issuedAt, dueAt, (state as?
LoanState.Returned)?.returnedAt, state.status)
private fun loanRecordToDomain(record: LoanRecord): Loan {
    val state = when (record.status) {
        "ACTIVE" → LoanState.Active
        "OVERDUE" → LoanState.Overdue
        "RETURNED" → LoanState.Returned(record.returnedAt ?: error("У возвращённой выдачи нет даты возврата"))
        else → error("Неизвестный статус выдачи: ${record.status}")
    }
    return Loan(record.id, record.readerId, record.publicationId, record.librarianId, record.issuedAt, record.dueAt, state)
}
```

core/src/jvmMain/kotlin/persistence/DatabaseJvm.kt

```
package persistence

import androidx.room3.Room
import androidx.room3.RoomDatabase
import java.io.File

fun databaseBuilder(databaseFile: File): RoomDatabase.Builder<LibraryDatabase> =
    Room.databaseBuilder<LibraryDatabase>(name = databaseFile.absolutePath)

fun defaultDatabaseFile(): File {
    val home = System.getProperty("user.home")
    val os = System.getProperty("os.name").lowercase()
    val path = when {
        os.contains("mac") → File(home, "Library/Application Support/CourseOopLibrary/library.db")
        os.contains("win") → File(
            System.getenv("LOCALAPPDATA") ?: File(home, "AppData/Local").absolutePath,
            "CourseOopLibrary/library.db",
        )
        else → File(home, ".local/share/course-oop-library/library.db")
    }
    path.parentFile.mkdirs()
    return path
}

fun openDefaultDatabase(): LibraryDatabase = buildDatabase(databaseBuilder(defaultDatabaseFile()))
```

app/src/commonMain/kotlin/App.kt

```
import androidx.compose.runtime.Composable
import di.ApplicationContainer
```

```
import ui.App as LibraryApp

@Composable
fun App(container: ApplicationContainer) {
    LibraryApp(container)
}
```

app/src/commonMain/kotlin/di/ApplicationContainer.kt

```
package di

import application.RoomLibraryService
import persistence.LibraryDatabase
import persistence.RoomRepositoryBundle
import persistence.openDefaultDatabase

/** Composition root. All collaborators are explicit dependencies; there are no global repositories. */
class ApplicationContainer(
    database: LibraryDatabase = openDefaultDatabase(),
) : AutoCloseable {
    private val closeables = LinkedHashSet<AutoCloseable>()
    private var closed = false
    val repositories: RoomRepositoryBundle = RoomRepositoryBundle(database)
    val service: RoomLibraryService = RoomLibraryService(repositories)

    fun register(closeable: AutoCloseable) {
        if (closed) closeable.close() else closeables += closeable
    }

    override fun close() {
        if (closed) return
        closed = true
        closeables.toList().asReversed().forEach(AutoCloseable::close)
        closeables.clear()
        repositories.database.close()
    }
}
```

app/src/commonMain/kotlin/ui/LibraryViewModels.kt

```
package ui

import domain.Book
import domain.IdGenerator
import domain.LibraryBranch
import domain.LibraryService
import domain.Loan
import domain.Librarian
import domain.Periodical
import domain.Publication
import domain.Reader
import domain.UuidIdGenerator
import kotlinx.coroutines.CancellationException
import kotlinx.coroutines.CoroutineScope
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.SupervisorJob
import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.SharingStarted
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.flow.catch
```

```
import kotlinx.coroutines.flow.combine
import kotlinx.coroutines.flow.first
import kotlinx.coroutines.flow.map
import kotlinx.coroutines.flow.stateIn
import kotlinx.coroutines.cancel
import kotlinx.coroutines.delay
import kotlinx.coroutines.isActive
import kotlinx.coroutines.launch
import persistence.BranchRepository
import persistence.LibrarianRepository
import persistence.LoanRepository
import persistence.PublicationRepository
import persistence.ReaderRepository

private inline fun <T> runCatchingCancellable(block: () -> T): Result<T> = try {
    Result.success(block())
} catch (error: CancellationException) {
    throw error
} catch (error: Throwable) {
    Result.failure(error)
}

data class CollectionUiState<T>(
    val items: List<T> = emptyList(),
    val isLoading: Boolean = true,
    val error: String? = null,
)

class ReadersViewModel(
    private val repository: ReaderRepository,
    private val ids: IdGenerator = UuidIdGenerator(),
    private val service: LibraryService? = null,
    private val scope: CoroutineScope = CoroutineScope(SupervisorJob() + Dispatchers.Default),
) : AutoCloseable {
    private val errors = MutableStateFlow<String?>(null)
    private val searchQuery = MutableStateFlow("")
    val query: StateFlow<String> = searchQuery.asStateFlow()
    val state: StateFlow<CollectionUiState<Reader>> = combine(repository.readers, searchQuery) { readers, query ->
        CollectionUiState(
            items = readers.filter { it.fullName.contains(query.trim(), ignoreCase = true) },
            isLoading = false,
        )
    }
        .catch { emit(CollectionUiState(isLoading = false, error = it.message ?: "Не удалось загрузить читателей")) }
        .combine(errors) { value, error -> value.copy(error = error ?: value.error) }
        .stateIn(scope, SharingStarted.Eagerly, CollectionUiState())

    fun setSearchQuery(value: String) {
        searchQuery.value = value
    }

    fun add(fullName: String, birthYear: String, address: String, organization: String): Boolean {
        val year = birthYear.toIntOrNull()
        if (fullName.isBlank() || year == null || address.isBlank() || organization.isBlank()) {
            errors.value = "Введите имя, корректный год рождения, адрес и организацию"
            return false
        }
        errors.value = null
        scope.launch {
            runCatchingCancellable { repository.save(Reader(ids.nextId(), fullName, year, address, organization)) }
                .onFailure { errors.value = it.message ?: "Не удалось сохранить читателя" }
```

```
        }
        return true
    }

    fun delete(reader: Reader) {
        scope.launch {
            runCatchingCancellable { service?.deleteReader(reader.id) ?: repository.delete(reader.id) }
                .onSuccess { errors.value = null }
                .onFailure { errors.value = it.message ?: "Не удалось удалить читателя" }
        }
    }
    override fun close() = scope.cancel()
}

class PublicationsViewModel(
    private val repository: PublicationRepository,
    private val branchesRepository: BranchRepository,
    private val ids: IdGenerator = UuidIdGenerator(),
    private val service: LibraryService? = null,
    private val scope: CoroutineScope = CoroutineScope(SupervisorJob() + Dispatchers.Default),
) : AutoCloseable {
    private val errors = MutableStateFlow<String?>(null)
    private val searchQuery = MutableStateFlow("")
    val query: StateFlow<String> = searchQuery.asStateFlow()
    val state: StateFlow<CollectionUiState<Publication>> = combine(repository.publications, searchQuery) { publications, query ->
        val normalized = query.trim()
        CollectionUiState(
            items = publications.filter {
                normalized.isBlank() || it.title.contains(normalized, ignoreCase = true) || it.authors.contains(normalized, ignoreCase =
true)
            },
            isLoading = false,
        )
    }
        .catch { emit(CollectionUiState(isLoading = false, error = it.message ?: "Не удалось загрузить издания")) }
        .combine(errors) { value, error -> value.copy(error = error ?: value.error) }
        .stateIn(scope, SharingStarted.Eagerly, CollectionUiState())

    val branches: StateFlow<CollectionUiState<LibraryBranch>> = branchesRepository.branches
        .map { CollectionUiState(items = it, isLoading = false) }
        .catch { emit(CollectionUiState(isLoading = false, error = it.message ?: "Не удалось загрузить филиалы")) }
        .stateIn(scope, SharingStarted.Eagerly, CollectionUiState())

    fun setSearchQuery(value: String) {
        searchQuery.value = value
    }

    fun addBook(title: String, authors: String, publisher: String, year: String, copies: String, branchId: String, isbn: String) {
        val parsedYear = year.toIntOrNull()
        val parsedCopies = copies.toIntOrNull()
        if (title.isBlank() || authors.isBlank() || publisher.isBlank() || parsedYear == null || parsedCopies == null ||
branchId.isBlank()) {
            errors.value = "Введите название, авторов, издателя, год, число экземпляров и филиал"
            return
        }
        errors.value = null
        scope.launch {
            runCatchingCancellable { repository.save(Book(ids.nextId(), title, authors, publisher, parsedYear, parsedCopies, parsedCopies,
branchId, isbn.ifBlank { null })) }
                .onFailure { errors.value = it.message ?: "Не удалось сохранить издание" }
        }
```

```
    }

    fun addPeriodical(title: String, editors: String, publisher: String, year: String, issue: String, copies: String, branchId: String) {
        val parsedYear = year.toIntOrNull()
        val parsedIssue = issue.toIntOrNull()
        val parsedCopies = copies.toIntOrNull()
        if (title.isBlank() || editors.isBlank() || publisher.isBlank() || parsedYear == null || parsedIssue == null || parsedCopies ==
null || branchId.isBlank()) {
            errors.value = "Введите название, редакторов, издателя, год, номер выпуска, число экземпляров и филиал"
            return
        }
        errors.value = null
        scope.launch {
            runCatchingCancellable { repository.save(Periodical(ids.nextId(), title, editors, publisher, parsedYear, parsedCopies,
parsedCopies, branchId, parsedIssue)) }
                .onFailure { errors.value = it.message ?: "Не удалось сохранить периодическое издание" }
        }
    }

    fun delete(publication: Publication) {
        scope.launch {
            runCatchingCancellable { service?.deletePublication(publication.id) ?: repository.delete(publication.id) }
                .onSuccess { errors.value = null }
                .onFailure { errors.value = it.message ?: "Не удалось удалить издание" }
        }
    }
    override fun close() = scope.cancel()
}

data class LoanRow(val loan: Loan, val readerName: String, val publicationTitle: String)

class LoansViewModel(
    private val service: LibraryService,
    private val loansRepository: LoanRepository,
    private val readersRepository: ReaderRepository,
    private val publicationsRepository: PublicationRepository,
    private val librariansRepository: LibrarianRepository,
    private val scope: CoroutineScope = CoroutineScope(SupervisorJob() + Dispatchers.Default),
    private val overdueRefreshIntervalMillis: Long = 60_000L,
    private val currentTimeMillis: () -> Long = ::systemTimeMillis,
) : AutoCloseable {
    private val operationErrors = MutableStateFlow<String?>(null)
    private val refreshErrors = MutableStateFlow<String?>(null)
    val state: StateFlow<CollectionUiState<LoanRow>> = combine(
        loansRepository.loans,
        readersRepository.readers,
        publicationsRepository.publications,
    ) { loans, readers, publications ->
        val readerNames = readers.associateBy(Reader::id)
        val titles = publications.associateBy(Publication::id)
        CollectionUiState(
            items = loans.map { LoanRow(it, readerNames[it.readerId]?.fullName ?: "Неизвестный читатель",
titles[it.publicationId]?.title ?: "Неизвестное издание") },
            isLoading = false,
        )
    }.catch { emit(CollectionUiState(isLoading = false, error = it.message ?: "Не удалось загрузить выдачи")) }
        .combine(refreshErrors) { value, error -> value.copy(error = error ?: value.error) }
        .combine(operationErrors) { value, error -> value.copy(error = error ?: value.error) }
        .stateIn(scope, SharingStarted.Eagerly, CollectionUiState())

    init {
```

```
        require(overdueRefreshIntervalMillis > 0L) { "Интервал обновления просроченных выдач должен быть положительным" }
        scope.launch {
            while (isActive) {
                delay(overdueRefreshIntervalMillis)
                runCatchingCancellable { loansRepository.refreshOverdue(currentTimeMillis()) }
                    .onSuccess { refreshErrors.value = null }
                    .onFailure { refreshErrors.value = it.message ?: "Не удалось обновить просроченные выдачи" }
            }
        }
    }

    fun issue(reader: Reader?, publication: Publication?) {
        if (reader == null || publication == null) {
            operationErrors.value = "Выберите читателя по имени и издание по названию"
            return
        }
        scope.launch {
            runCatchingCancellable {
                val librarian = librariansRepository.librarians.first().firstOrNull()
                    ?: error("Зарегистрируйте библиотекаря перед выдачей")
                @OptIn(kotlin.time.ExperimentalTime::class)
                val now = kotlin.time.Clock.System.now().toEpochMilliseconds()
                service.issueLoan(reader.id, publication.id, librarian.id, now)
            }
                .onSuccess { operationErrors.value = null }
                .onFailure { operationErrors.value = it.message ?: "Не удалось оформить выдачу" }
        }
    }

    fun returnLoan(loan: Loan) {
        scope.launch {
            runCatchingCancellable {
                @OptIn(kotlin.time.ExperimentalTime::class)
                service.returnLoan(loan.id, kotlin.time.Clock.System.now().toEpochMilliseconds())
            }
                .onSuccess { operationErrors.value = null }
                .onFailure { operationErrors.value = it.message ?: "Не удалось оформить возврат" }
        }
    }
    override fun close() = scope.cancel()
}

@OptIn(kotlin.time.ExperimentalTime::class)
private fun systemTimeMillis(): Long = kotlin.time.Clock.System.now().toEpochMilliseconds()

data class AdministrationUiState(
    val isBusy: Boolean = false,
    val message: String? = null,
    val error: String? = null,
)

class AdministrationViewModel(
    private val service: LibraryService,
    private val scope: CoroutineScope = CoroutineScope(SupervisorJob() + Dispatchers.Default),
) : AutoCloseable {
    private val _state = MutableStateFlow(AdministrationUiState())
    val state: StateFlow<AdministrationUiState> = _state.asStateFlow()

    fun seedDemo() {
        scope.launch {
            _state.value = AdministrationUiState(isBusy = true)
```

```
            try {
                runCatchingCancellable { service.seedDemo() }
                    .onSuccess { inserted ->
                        _state.value = AdministrationUiState(message = if (inserted) "Демонстрационные данные добавлены" else
"Демонстрационные данные уже существуют")
                    }
                    .onFailure { _state.value = AdministrationUiState(error = it.message ?: "Не удалось добавить демонстрационные
данные") }
            } finally {
                clearBusyAfterStandaloneCancellation()
            }
        }
    }

    fun clearAll() {
        scope.launch {
            _state.value = AdministrationUiState(isBusy = true)
            try {
                runCatchingCancellable { service.clearAll() }
                    .onSuccess { result -> _state.value = AdministrationUiState(message = "Удалено записей: ${result.totalDeleted}") }
                    .onFailure { _state.value = AdministrationUiState(error = it.message ?: "Не удалось очистить данные") }
            } finally {
                clearBusyAfterStandaloneCancellation()
            }
        }
    }

    private fun clearBusyAfterStandaloneCancellation() {
        if (scope.isActive && _state.value.isBusy) {
            _state.value = _state.value.copy(isBusy = false)
        }
    }

    override fun close() = scope.cancel()
}
```

app/src/commonMain/kotlin/ui/LibraryScreen.kt

```
package ui

import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.Row
import androidx.compose.foundation.layout.Spacer
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.fillMaxWidth
import androidx.compose.foundation.layout.height
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.layout.width
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.material3.Button
import androidx.compose.material3.Card
import androidx.compose.material3.MaterialTheme
import androidx.compose.material3.OutlinedTextField
import androidx.compose.material3.Scaffold
import androidx.compose.material3.Tab
import androidx.compose.material3.TabRow
import androidx.compose.material3.Text
import androidx.compose.material3.TextButton
import androidx.compose.runtime.Composable
import androidx.compose.runtime.DisposableEffect
```

```
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue
import androidx.compose.runtime.mutableIntStateOf
import androidx.compose.runtime.mutableStateOf
import androidx.compose.runtime.remember
import androidx.compose.runtime.setValue
import androidx.compose.ui.Modifier
import androidx.compose.ui.unit.dp
import di.ApplicationContainer
import domain.LoanState
import domain.Publication
import domain.Reader

@Composable
fun App(container: ApplicationContainer) {
    val readers = remember(container) { ReadersViewModel(container.repositories.readers, service = container.service) }
    val publications = remember(container) {
        PublicationsViewModel(container.repositories.publications, container.repositories.branches, service = container.service)
    }
    val loans = remember(container) {
        LoansViewModel(container.service, container.repositories.loans, container.repositories.readers,
container.repositories.publications, container.repositories.librarians)
    }
    val administration = remember(container) { AdministrationViewModel(container.service) }
    DisposableEffect(container) {
        container.register(readers)
        container.register(publications)
        container.register(loans)
        container.register(administration)
        onDispose {
            readers.close()
            publications.close()
            loans.close()
            administration.close()
        }
    }
    LibraryScreen(readers, publications, loans, administration)
}

@Composable
private fun LibraryScreen(
    readersVm: ReadersViewModel,
    publicationsVm: PublicationsViewModel,
    loansVm: LoansViewModel,
    administrationVm: AdministrationViewModel,
) {
    var tab by remember { mutableIntStateOf(0) }
    Scaffold { padding ->
        Column(Modifier.fillMaxSize().padding(padding).padding(16.dp)) {
            Text("Библиотека курса ООП", style = MaterialTheme.typography.headlineMedium)
            Spacer(Modifier.height(8.dp))
            TabRow(tab) {
                listOf("Читатели", "Издания", "Выдачи", "Архитектура").forEachIndexed { index, title ->
                    Tab(selected = tab == index, onClick = { tab = index }, text = { Text(title) })
                }
            }
            Spacer(Modifier.height(12.dp))
            when (tab) {
                0 -> ReadersPane(readersVm)
                1 -> PublicationsPane(publicationsVm)
                2 -> LoansPane(loansVm, readersVm.state.collectAsState().value.items, publicationsVm.state.collectAsState().value.items)
```

```
                else -> ArchitecturePane(administrationVm)
            }
        }
    }
}

@Composable
private fun ReadersPane(vm: ReadersViewModel) {
    val state by vm.state.collectAsState()
    val query by vm.query.collectAsState()
    var name by remember { mutableStateOf("") }
    var year by remember { mutableStateOf("") }
    var address by remember { mutableStateOf("") }
    var organization by remember { mutableStateOf("") }
    LazyColumn(
        modifier = Modifier.fillMaxSize(),
        verticalArrangement = Arrangement.spacedBy(6.dp),
    ) {
        item {
            OutlinedTextField(
                value = query,
                onValueChange = vm::setSearchQuery,
                modifier = Modifier.fillMaxWidth(),
                label = { Text("Поиск читателей по имени") },
            )
        }
        item {
            FormRow {
                Row(Modifier.fillMaxWidth(), horizontalArrangement = Arrangement.spacedBy(8.dp)) {
                    OutlinedTextField(name, { name = it }, Modifier.weight(1f), label = { Text("Полное имя") })
                    OutlinedTextField(year, { year = it }, Modifier.weight(1f), label = { Text("Год рождения") })
                }
                Row(Modifier.fillMaxWidth(), horizontalArrangement = Arrangement.spacedBy(8.dp)) {
                    OutlinedTextField(address, { address = it }, Modifier.weight(1f), label = { Text("Адрес") })
                    OutlinedTextField(organization, { organization = it }, Modifier.weight(1f), label = { Text("Организация") })
                }
                Button(onClick = { vm.add(name, year, address, organization) }) { Text("Добавить читателя") }
            }
        }
        item { Status(state.isLoading, state.error, state.items.isEmpty(), "Читателей пока нет") }
        items(state.items) { reader ->
            Card(Modifier.fillMaxWidth()) {
                Row(Modifier.fillMaxWidth().padding(8.dp), horizontalArrangement = Arrangement.SpaceBetween) {
                    Text("${reader.fullName} · ${reader.organization}", Modifier.weight(1f).padding(4.dp))
                    TextButton(onClick = { vm.delete(reader) }) { Text("Удалить") }
                }
            }
        }
    }
}

@Composable
private fun PublicationsPane(vm: PublicationsViewModel) {
    val state by vm.state.collectAsState()
    val query by vm.query.collectAsState()
    val branches by vm.branches.collectAsState()
    var title by remember { mutableStateOf("") }
    var authors by remember { mutableStateOf("") }
    var publisher by remember { mutableStateOf("") }
    var year by remember { mutableStateOf("") }
    var copies by remember { mutableStateOf("") }
```

```
    var selectedBranch by remember { mutableStateOf<domain.LibraryBranch?>(null) }
    var isbn by remember { mutableStateOf("") }
    var issue by remember { mutableStateOf("1") }
    LazyColumn(
        modifier = Modifier.fillMaxSize(),
        verticalArrangement = Arrangement.spacedBy(6.dp),
    ) {
        item {
            OutlinedTextField(
                value = query,
                onValueChange = vm::setSearchQuery,
                modifier = Modifier.fillMaxWidth(),
                label = { Text("Поиск изданий по названию или автору") },
            )
        }
        item {
            FormRow {
                Row(Modifier.fillMaxWidth(), horizontalArrangement = Arrangement.spacedBy(8.dp)) {
                    OutlinedTextField(title, { title = it }, Modifier.weight(1f), label = { Text("Название") })
                    OutlinedTextField(authors, { authors = it }, Modifier.weight(1f), label = { Text("Авторы / редакторы") })
                }
                Row(Modifier.fillMaxWidth(), horizontalArrangement = Arrangement.spacedBy(8.dp)) {
                    OutlinedTextField(publisher, { publisher = it }, Modifier.weight(1f), label = { Text("Издатель") })
                    OutlinedTextField(year, { year = it }, Modifier.weight(1f), label = { Text("Год издания") })
                }
                Row(Modifier.fillMaxWidth(), horizontalArrangement = Arrangement.spacedBy(8.dp)) {
                    OutlinedTextField(copies, { copies = it }, Modifier.weight(1f), label = { Text("Экземпляры") })
                    OutlinedTextField(isbn, { isbn = it }, Modifier.weight(1f), label = { Text("ISBN (необязательно)") })
                }
                Row(Modifier.fillMaxWidth(), horizontalArrangement = Arrangement.spacedBy(8.dp)) {
                    Column(Modifier.weight(1f)) {
                        Text("Филиал", style = MaterialTheme.typography.titleSmall)
                        Row(horizontalArrangement = Arrangement.spacedBy(4.dp)) {
                            branches.items.forEach { branch ->
                                TextButton(onClick = { selectedBranch = branch }) { Text(branch.name) }
                            }
                        }
                        Text(selectedBranch?.name ?: "Филиал не выбран")
                    }
                    OutlinedTextField(issue, { issue = it }, Modifier.weight(1f), label = { Text("Номер выпуска") })
                }
                Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
                    Button(onClick = { vm.addBook(title, authors, publisher, year, copies, selectedBranch?.id ?: "", isbn) })
{ Text("Добавить книгу") }
                    Button(onClick = { vm.addPeriodical(title, authors, publisher, year, issue, copies, selectedBranch?.id ?: "") })
{ Text("Добавить периодическое издание") }
                }
            }
        }
        item { Status(state.isLoading, state.error ?: branches.error, state.items.isEmpty(), "Изданий пока нет") }
        items(state.items) { publication ->
            Card(Modifier.fillMaxWidth()) {
                Row(Modifier.fillMaxWidth().padding(8.dp), horizontalArrangement = Arrangement.SpaceBetween) {
                    Text("${publication.title} · ${publication.authors} · ${publication.availableCopies}/${publication.totalCopies}",
Modifier.weight(1f).padding(4.dp))
                    TextButton(onClick = { vm.delete(publication) }) { Text("Удалить") }
                }
            }
        }
    }
}
```

```
@Composable
private fun LoansPane(vm: LoansViewModel, readers: List<Reader>, publications: List<Publication>) {
    val state by vm.state.collectAsState()
    var selectedReader by remember { mutableStateOf<Reader?>(null) }
    var selectedPublication by remember { mutableStateOf<Publication?>(null) }
    LazyColumn(
        modifier = Modifier.fillMaxSize(),
        verticalArrangement = Arrangement.spacedBy(6.dp),
    ) {
        item {
            Column(verticalArrangement = Arrangement.spacedBy(8.dp)) {
                Text("Выберите читателя по имени и издание по названию; технические идентификаторы скрыты внутри приложения.")
                Text("Читатели", style = MaterialTheme.typography.titleSmall)
                Row(horizontalArrangement = Arrangement.spacedBy(4.dp)) {
                    readers.forEach { reader ->
                        TextButton(onClick = { selectedReader = reader }) { Text(reader.fullName) }
                    }
                }
                Text("Издания", style = MaterialTheme.typography.titleSmall)
                Row(horizontalArrangement = Arrangement.spacedBy(4.dp)) {
                    publications.filter { it.availableCopies > 0 }.forEach { publication ->
                        TextButton(onClick = { selectedPublication = publication }) { Text(publication.title) }
                    }
                }
                Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
                    Text(selectedReader?.fullName ?: "Читатель не выбран")
                    Text(selectedPublication?.title ?: "Издание не выбрано")
                    Button(onClick = { vm.issue(selectedReader, selectedPublication) }) { Text("Выдать") }
                }
            }
        }
        item { Status(state.isLoading, state.error, state.items.isEmpty(), "Выдач пока нет") }
        items(state.items) { row ->
            Card(Modifier.fillMaxWidth()) {
                Row(Modifier.fillMaxWidth().padding(12.dp), horizontalArrangement = Arrangement.SpaceBetween) {
                    Text("${row.readerName} · ${row.publicationTitle}")
                    if (row.loan.state !is LoanState.Returned) Button(onClick = { vm.returnLoan(row.loan) }) { Text("Вернуть") }
                }
            }
        }
    }
}

@Composable
private fun ArchitecturePane(vm: AdministrationViewModel) {
    val state by vm.state.collectAsState()
    LazyColumn(
        modifier = Modifier.fillMaxSize(),
        verticalArrangement = Arrangement.spacedBy(6.dp),
    ) {
        item {
            Card(Modifier.fillMaxWidth()) {
                Column(Modifier.padding(10.dp), verticalArrangement = Arrangement.spacedBy(3.dp)) {
                    Text("ООП-архитектура", style = MaterialTheme.typography.titleLarge)
                    Text("Наследование: LibraryEntity → Person → Reader / Librarian")
                    Text("Наследование: LibraryEntity → Publication → Book / Periodical")
                    Text("Читатель ⇔ издание: связь многие-ко-многим через Loan")
                    Text("Конвейер Room: Compose → ViewModel → LibraryService → Repository → DAO → SQLite")
                    Text("Паттерны: Repository · MVVM · Strategy · Factory · Observer · State · Facade")
                }
```

```
            }
        }
        item {
            Card(Modifier.fillMaxWidth()) {
                Column(Modifier.padding(10.dp), verticalArrangement = Arrangement.spacedBy(6.dp)) {
                    Text("Данные приложения", style = MaterialTheme.typography.titleMedium)
                    Text("Заполнение демонстрационными данными идемпотентно. Очистка транзакционна и удаляет выдачи перед зависимыми
записями.")
                    Row(horizontalArrangement = Arrangement.spacedBy(8.dp)) {
                        Button(enabled = !state.isBusy, onClick = vm::seedDemo) { Text("Добавить демоданные") }
                        Button(enabled = !state.isBusy, onClick = vm::clearAll) { Text("Очистить все данные") }
                    }
                    when {
                        state.isBusy -> Text("Выполняется…")
                        state.error != null -> Text(state.error!!, color = MaterialTheme.colorScheme.error)
                        state.message != null -> Text(state.message!!)
                    }
                }
            }
        }
    }
}

@Composable
private fun FormRow(content: @Composable () -> Unit) {
    Column(verticalArrangement = Arrangement.spacedBy(6.dp), modifier = Modifier.fillMaxWidth()) { content() }
}

@Composable
private fun Status(loading: Boolean, error: String?, empty: Boolean, emptyText: String) {
    when {
        loading -> Text("Загрузка…")
        error != null -> Text(error, color = MaterialTheme.colorScheme.error)
        empty -> Text(emptyText, color = MaterialTheme.colorScheme.onSurfaceVariant)
    }
}
```

app/src/jvmMain/kotlin/main.kt

```
import androidx.compose.ui.window.Window
import androidx.compose.ui.window.application
import di.ApplicationContainer
import ui.App

fun main() {
    val container = ApplicationContainer()
    application {
        Window(
            onCloseRequest = {
                container.close()
                exitApplication()
            },
            title = "Библиотека курса ООП",
        ) {
            App(container)
        }
    }
}
```